

Java

Introduzione a Java

Cos'è Java, dove viene usato e come iniziare a scrivere programmi semplici.

Cos'è Java

Java è un linguaggio di programmazione usato per creare programmi che possono funzionare su computer diversi: Windows, macOS, Linux, server, telefoni Android e molti altri ambienti.

Lo trovi in applicazioni aziendali, servizi web, app Android, strumenti per banche, università, negozi online e grandi sistemi che devono restare stabili nel tempo.

Un programma Java può essere semplice come questo:

```
public class Saluto {  
    public static void main(String[] args) {  
        System.out.println("Ciao, Java!");  
    }  
}
```

Output:

```
Ciao, Java!
```

All'inizio alcune parole sembrano lunghe: `public`, `class`, `static`, `main`. Non devi capirle tutte subito. In questa guida le incontreremo un pezzo alla volta.

Perché Java è diverso da molti linguaggi

Java non viene eseguito direttamente come un file di testo. Prima viene **compilato**, cioè trasformato in un formato intermedio.

Poi entra in gioco la **JVM**, la Java Virtual Machine. La JVM è il programma che legge quel formato intermedio ed esegue il tuo codice sul computer.

Il percorso è questo:

1. scrivi un file `.java`
2. lo compili con `javac`
3. Java produce un file `.class`
4. esegui quel file con `java`

Questa idea rende Java molto portabile: lo stesso programma può funzionare su sistemi diversi, se hanno una JVM compatibile.

Dove si usa Java

Java è usato soprattutto quando servono programmi affidabili, ordinati e facili da mantenere nel tempo.

Alcuni esempi:

- applicazioni web e servizi lato server
- app Android
- programmi gestionali
- sistemi bancari e assicurativi
- strumenti per aziende e università
- applicazioni con database

Java non è sempre il linguaggio più breve da scrivere. In cambio ti spinge a organizzare bene il codice. Questa caratteristica diventa utile quando i programmi crescono.

Cosa imparerai in questa guida

Partiremo dall'ambiente di lavoro: installare Java, scrivere un primo programma, compilarlo ed eseguirlo.

Poi vedremo le basi:

- variabili e tipi di dato
- operatori
- input e output
- condizioni e cicli
- array e collezioni
- metodi

Quando questi pezzi saranno familiari, passeremo alla programmazione a oggetti: classi, oggetti, costruttori, incapsulamento, ereditarietà e interfacce.

Alla fine incontrerai anche errori, test, strumenti di build e alcuni concetti moderni come generics, lambda, stream, record ed enum.

Come studiare questa guida

Java si capisce meglio scrivendo programmi piccoli. Non cercare di memorizzare tutto al primo passaggio.

Per ogni pagina prova a fare così:

1. leggi l'idea principale
2. copia l'esempio in un file
3. eseguilo
4. cambia un valore
5. osserva cosa succede

Se un errore compare nel terminale, non è un fallimento. In Java gli errori fanno parte del percorso: spesso indicano una riga precisa e ti aiutano a capire cosa correggere.

Un primo assaggio

Questo programma salva un nome in una variabile e lo usa in un messaggio:

```
public class Presentazione {  
    public static void main(String[] args) {  
        String nome = "Luca";  
        System.out.println("Ciao, " + nome + "!");  
    }  
}
```

Output:

```
Ciao, Luca!
```

Qui succedono due cose semplici:

- `String nome = "Luca";` salva un testo nella variabile `nome`
- `System.out.println(...)` stampa un messaggio nel terminale

Nelle prossime pagine renderemo chiaro ogni pezzo, senza correre.

Generics

Come usare tipi parametrizzati per collezioni e classi riutilizzabili.

Il problema che risolvono

Hai già visto codice come questo:

```
ArrayList nomi = new ArrayList<>();
```

La parte `String` tra parentesi angolari dice che la lista contiene stringhe.

Questa idea si chiama **generics**.

Tipi più sicuri

Senza generics, una lista potrebbe contenere di tutto: testi, numeri, oggetti diversi.

Con i generics, Java controlla il tipo.

```
ArrayList nomi = new ArrayList<>();

nomi.add("Luca");
nomi.add("Sara");
// nomi.add(10); // errore
```

`10` è un `int`, non una `String`. Java lo segnala subito.

Generics nelle collezioni

Esempi comuni:

```
ArrayList nomi = new ArrayList<>();
ArrayList numeri = new ArrayList<>();
HashMap prezzi = new HashMap<>();
HashSet parole = new HashSet<>();
```

`Integer` è la versione oggetto di `int`. Nelle collezioni Java si usano tipi oggetto, non primitivi.

Altri esempi:

- `Double` per `double`
- `Boolean` per `boolean`
- `Character` per `char`

Creare una classe generica

Puoi creare anche le tue classi generiche.

```
public class Scatola {
    private T valore;

    public Scatola(T valore) {
        this.valore = valore;
    }

    public T getValore() {
        return valore;
    }
}
```

`T` è un parametro di tipo. Significa: il tipo verrà scelto quando userai la classe.

Usare la classe generica

```
public class Esempio {  
    public static void main(String[] args) {  
        Scatola scatolaNome = new Scatola<>("Luca");  
        Scatola scatolaNumero = new Scatola<>(42);  
  
        System.out.println(scatolaNome.getValore());  
        System.out.println(scatolaNumero.getValore());  
    }  
}
```

Output:

```
Luca  
42
```

La stessa classe `Scatola` funziona con tipi diversi, ma resta controllata.

Perche non usare Object

Potresti pensare di usare `Object`, il tipo piu generale.

```
public class Scatola {  
    private Object valore;  
}
```

Il problema e che poi perdi informazioni sul tipo e devi fare cast manuali.

I generics evitano molti cast e molti errori.

Regola pratica

Usa i generics quando una classe o una collezione deve lavorare con tipi diversi, ma vuoi mantenere controlli chiari.

Nella maggior parte dei programmi iniziali li incontrerai soprattutto con collezioni come `ArrayList`, `HashMap` e `HashSet`.

Lambda

Come scrivere funzioni brevi da passare ad altri metodi.

A cosa servono le lambda

Una **lambda** è un modo breve per scrivere una piccola azione.

La usi quando un metodo ti chiede:

"Che cosa devo fare con questo valore?"

Per esempio, una lista può chiederti cosa fare con ogni nome. Tu puoi rispondere con una lambda:

```
nome -> System.out.println(nome)
```

Si legge così:

"prendi un `nome` e stampalo".

La freccia `->` separa due parti:

- a sinistra c'è il valore che arriva
- a destra c'è l'azione da eseguire

La forma più semplice

La forma più comune è questa:

```
parametro -> azione
```

Il `parametro` è il valore che Java passa alla lambda.

L' `azione` è il codice che vuoi eseguire con quel valore.

Esempio:

```
nome -> System.out.println(nome)
```

Qui `nome` non è una variabile creata prima da noi. È il nome che diamo al valore ricevuto dalla lambda.

Usare una lambda con `forEach`

Vediamo un esempio completo con una lista di nomi.

```
public class EsempioLambda {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        nomi.forEach(nome -> System.out.println(nome));  
    }  
}
```

Output:

```
Luca  
Sara  
Mina
```

`forEach` significa "per ciascun elemento".

Quindi questa riga:

```
nomi.forEach(nome -> System.out.println(nome));
```

vuol dire:

"per ogni nome nella lista `nomi` , stampa quel nome".

Java prende un elemento alla volta dalla lista. Ogni elemento viene chiamato `nome` dentro la lambda.

Confronto con il for-each

La stessa cosa si può scrivere anche con un ciclo `for-each` .

```
for (String nome : nomi) {  
    System.out.println(nome);  
}
```

Con una lambda diventa:

```
nomi.forEach(nome -> System.out.println(nome));
```

I due esempi fanno la stessa cosa.

All'inizio il ciclo `for-each` è spesso più facile da leggere. La lambda diventa utile quando usi metodi che vogliono ricevere una piccola azione, come `forEach` , `sort` , `filter` o `map` .

Lambda con più righe

Se l'azione è molto breve, puoi scriverla dopo la freccia.

Se invece servono più righe, usa le parentesi graffe.

```
nomi.forEach(nome -> {  
    String maiuscolo = nome.toUpperCase();  
    System.out.println(maiuscolo);  
});
```

Output:

```
LUCA  
SARA  
MINA
```

Qui succedono due cose per ogni nome:

1. Java crea una versione in maiuscolo.
2. Poi stampa il risultato.

Attenzione: se usi le parentesi graffe, il codice dentro la lambda deve essere scritto come un normale blocco Java.

Lambda che restituisce un valore

Una lambda può anche produrre un risultato.

Per esempio:

```
numero -> numero * 2
```

Si legge:

"prendi `numero` e restituisci `numero * 2`".

In una lambda breve non devi scrivere `return`.

```
Operazione doppio = numero -> numero * 2;
```

Questa lambda prende un numero e produce il suo doppio.

Interfacce funzionali

Una lambda non vive da sola. Java deve sapere che tipo di azione rappresenta.

Per questo una lambda si usa quando Java si aspetta un'interfaccia con **un solo metodo astratto**.

Questa interfaccia si chiama **interfaccia funzionale**.

Esempio:

```
public interface Operazione {  
    int esegui(int numero);  
}
```

Questa interfaccia dice:

"un'operazione prende un `int` e restituisce un `int`".

Ora possiamo creare un'operazione con una lambda.

```
Operazione doppio = numero -> numero * 2;  
  
System.out.println(doppio.esegui(5));
```

Output:

```
10
```

La lambda:

```
numero -> numero * 2
```

diventa il corpo del metodo `esegui`.

Quando chiami:

```
doppio.esegui(5)
```

Java mette `5` al posto di `numero` e calcola `5 * 2`.

Lambda con due parametri

Una lambda puo ricevere anche piu valori.

In quel caso metti i parametri tra parentesi.

```
(a, b) -> a.compareTo(b)
```

Qui la lambda riceve due valori: `a` e `b` .

Vediamola dentro `sort` , che ordina una lista.

```
public class OrdinaNomi {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Sara");  
        nomi.add("Luca");  
        nomi.add("Mina");  
  
        nomi.sort((a, b) -> a.compareTo(b));  
  
        System.out.println(nomi);  
    }  
}
```

Output:

```
[Luca, Mina, Sara]
```

`sort` deve sapere come confrontare due elementi.

La lambda:

```
(a, b) -> a.compareTo(b)
```

dice a Java:

"confronta `a` con `b` usando l'ordine naturale delle stringhe".

Quando usarle

Le lambda sono utili quando:

- l'azione è breve
- il codice resta facile da leggere
- stai passando un comportamento a un metodo
- usi collezioni, ordinamenti o stream

Evita lambda troppo lunghe. Se dentro la lambda ci sono molte righe, spesso è meglio creare un metodo con un nome chiaro.

Da ricordare

- Una lambda descrive una piccola azione.
 - La freccia `->` significa "usa questo valore per fare questa cosa".
 - A sinistra della freccia ci sono i parametri.
 - A destra della freccia c'è il codice da eseguire.
 - Le lambda sono utili quando un metodo chiede un comportamento da applicare.
-

Record ed enum

Come rappresentare dati compatti e insiemi di valori fissi.

Due strumenti per dati semplici

Java offre strumenti che rendono più comodo rappresentare alcuni dati.

In questa pagina vediamo:

- `enum`, per valori fissi
- `record`, per oggetti semplici e immutabili

Sono concetti più moderni rispetto alle basi, ma molto utili quando il modello dei dati è chiaro.

enum

Un `enum` rappresenta un insieme chiuso di valori possibili.

Esempio:

```
public enum Giorno {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
}
```

Una variabile di tipo `Giorno` può contenere solo uno di questi valori.

```
Giorno oggi = Giorno.LUNEDI;
```

Perche usare enum

Senza enum potresti usare stringhe:

```
String stato = "pagato";
```

Ma potresti anche sbagliare:

```
String stato = "pagatto";
```

Con un enum, Java controlla i valori validi.

```
public enum StatoOrdine {  
    NUOVO,  
    PAGATO,  
    SPEDITO,  
    CONSEGNATO  
}
```

Uso:

```
StatoOrdine stato = StatoOrdine.PAGATO;
```

enum con switch

```
StatoOrdine stato = StatoOrdine.SPEDITO;  
  
switch (stato) {  
    case NUOVO:  
        System.out.println("Ordine creato");  
        break;  
    case PAGATO:  
        System.out.println("Pagamento ricevuto");  
        break;  
    case SPEDITO:  
        System.out.println("Ordine in viaggio");  
        break;  
    case CONSEGNATO:  
        System.out.println("Ordine consegnato");  
        break;  
}
```

Gli enum sono ottimi quando hai un elenco limitato e conosciuto di scelte.

record

Un **record** serve a rappresentare dati semplici.

```
public record Prodotto(String nome, double prezzo) {  
}
```

Questa riga crea automaticamente:

- campi privati e finali
- costruttore
- metodi per leggere i valori
- `toString`
- `equals`
- `hashCode`

Usare un record

```
public class EsempioRecord {  
    public static void main(String[] args) {  
        Prodotto prodotto = new Prodotto("Quaderno", 2.50);  
  
        System.out.println(prodotto.nome());  
        System.out.println(prodotto.prezzo());  
        System.out.println(prodotto);  
    }  
}
```

Output:

```
Quaderno  
2.5  
Prodotto[nome=Quaderno, prezzo=2.5]
```

I metodi di lettura hanno lo stesso nome dei campi: `nome()` e `prezzo()`.

Record immutabili

Un record è pensato per dati immutabili: dopo la creazione, non cambi i campi.

Non fai:

```
prodotto.prezzo = 3.00; // non valido
```

Se ti serve un prodotto con prezzo diverso, crei un nuovo record.

Quando usarli

Usa `enum` quando un valore può essere scelto da un insieme fisso:

- giorni della settimana
- stati di un ordine
- livelli di priorità

Usa `record` quando vuoi trasportare dati semplici senza scrivere una classe lunga:

- prodotto con nome e prezzo
- punto con x e y
- persona con nome ed età

Se l'oggetto deve cambiare spesso o avere molta logica interna, una classe normale può essere più adatta.

Stream

Come trasformare e filtrare collezioni con una sintassi dichiarativa.

Cosa sono gli stream

Uno stream è un modo per lavorare su una sequenza di dati dichiarando cosa vuoi ottenere.

Invece di scrivere ogni passaggio con un ciclo, componi operazioni come:

- filtra
- trasforma
- raccogli il risultato

Gli stream non sostituiscono sempre i cicli. Sono uno strumento in più, utile quando il flusso di trasformazione è chiaro.

Da lista a stream

```
public class EsempioStream {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        List risultato = nomi.stream()  
            .filter(nome -> nome.length() > 4)  
            .toList();  
  
        System.out.println(risultato);  
    }  
}
```

Output:

```
[Luca, Sara]
```

`Mina` ha 4 caratteri, quindi viene esclusa dalla condizione `> 4`.

Collezione e stream

Una collezione, come `ArrayList`, contiene davvero i dati.

Uno stream è un flusso di lavoro su quei dati.

```
nomi.stream()
```

crea uno stream a partire dalla lista.

Alla fine, con `toList()`, raccogli il risultato in una nuova lista.

filter

`filter` tiene solo gli elementi che rispettano una condizione.

```
List numeri = List.of(3, 8, 1, 10, 5);

List grandi = numeri.stream()
    .filter(numero -> numero >= 5)
    .toList();

System.out.println(grandi);
```

Output:

```
[8, 10, 5]
```

La lambda deve restituire `true` o `false`.

map

`map` trasforma ogni elemento.

```
List nomi = List.of("luca", "sara", "mina");

List maiuscoli = nomi.stream()
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(maiuscoli);
```

Output:

```
[LUCA, SARA, MINA]
```

Il numero di elementi resta lo stesso, ma ogni elemento viene trasformato.

Combinare filter e map

```
List nomi = List.of("luca", "sara", "anna", "mario");

List risultato = nomi.stream()
    .filter(nome -> nome.length() == 4)
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(risultato);
```

Output:

```
[LUCA, SARA, ANNA]
```

Prima filtriamo i nomi lunghi 4 caratteri. Poi li trasformiamo in maiuscolo.

Quando usare uno stream

Uno stream è adatto quando stai descrivendo una trasformazione sui dati.

Un ciclo normale può essere migliore quando:

- la logica ha molti passaggi
- devi modificare variabili esterne
- il codice con stream diventa troppo difficile da leggere

Regola pratica

Se puoi leggere la catena ad alta voce, lo stream è probabilmente chiaro:

"prendi i nomi, tieni quelli lunghi 4, trasformali in maiuscolo, raccoglili in una lista".

Se devi fermarti a decifrare ogni pezzo, usa un ciclo più esplicito.

Commenti

Come scrivere note nel codice Java senza modificarne il comportamento.

A cosa servono i commenti

Un commento è una nota scritta nel codice. Java la ignora quando esegue il programma.

I commenti servono a spiegare un passaggio, lasciare un promemoria o rendere più chiara un'intenzione.

```
// Salviamo il nome dell'utente  
String nome = "Luca";
```

La riga che inizia con `//` non viene eseguita.

Commenti su una riga

Il commento più semplice inizia con `//` e continua fino alla fine della riga.

```
public class Esempio {  
    public static void main(String[] args) {  
        // Questo messaggio viene mostrato nel terminale  
        System.out.println("Ciao!");  
    }  
}
```

Puoi usarlo anche alla fine di una riga:

```
int eta = 20; // eta dell'utente
```

Usalo con misura. Se il nome della variabile è già chiaro, il commento può essere inutile.

Commenti su piu righe

Un commento su piu righe inizia con `/*` e finisce con `*/`.

```
/*
Questo programma calcola il totale
di un piccolo ordine.
*/
public class Totale {
    public static void main(String[] args) {
        double prezzo = 10.0;
        double spedizione = 3.5;
        System.out.println(prezzo + spedizione);
    }
}
```

E' utile quando la nota e piu lunga di una frase.

Commenti Javadoc

Java ha anche un tipo speciale di commento: il commento Javadoc.

Inizia con `/**` e finisce con `*/`.

```
/**
 * Calcola il totale aggiungendo prezzo e spedizione.
 */
public static double calcolaTotale(double prezzo, double spedizione) {
    return prezzo + spedizione;
}
```

Questi commenti possono essere usati da strumenti automatici per generare documentazione.

All'inizio non devi usarli sempre. Ti basta sapere che esistono e che di solito descrivono classi e metodi.

Commenti buoni e commenti inutili

Un commento e' utile quando spiega il perche di una scelta.

```
// Applichiamo lo sconto solo se il totale supera 50 euro
if (totale > 50) {
    totale = totale - 5;
}
```

Questo commento aiuta.

Questo invece aggiunge poco:

```
// Aumenta eta di 1
eta = eta + 1;
```

Il codice e gia abbastanza chiaro.

Disattivare codice per provare

Durante gli esercizi puoi commentare una riga per non eseguirla temporaneamente:

```
System.out.println("Prima riga");
// System.out.println("Seconda riga");
System.out.println("Terza riga");
```

Output:

```
Prima riga
Terza riga
```

Suggerimento: va bene farlo mentre studi. Prima di tenere un programma ordinato, pero, rimuovi il codice commentato che non serve piu.

Tipi di dato

I principali tipi di dato in Java e quando usare numeri, testi e valori booleani.

Perche esistono i tipi

Ogni variabile contiene un tipo di informazione. Un prezzo non e uguale a un nome. Una risposta vero/falso non e uguale a una frase.

In Java devi dichiarare il **tipo di dato** della variabile:

```
int eta = 20;  
String nome = "Luca";  
boolean attivo = true;
```

Il tipo dice a Java che cosa puo contenere quella variabile e quali operazioni hanno senso.

Numeri interi: int

`int` contiene numeri interi, senza virgola.

```
int eta = 20;  
int quantita = 3;  
int temperatura = -2;
```

Usalo per contatori, eta, quantita e punteggi.

```
int mele = 4;  
int pere = 2;  
int totale = mele + pere;  
  
System.out.println(totale);
```

Output:

6

Numeri decimali: double

`double` contiene numeri con la parte decimale.

```
double prezzo = 12.50;  
double altezza = 1.75;
```

In Java i decimali usano il punto, non la virgola.

```
double prezzo = 10.0;  
double spedizione = 4.90;  
double totale = prezzo + spedizione;  
  
System.out.println(totale);
```

Output:

14.9

Vero o falso: boolean

`boolean` contiene solo due valori:

```
true  
false
```

E' utile per rappresentare condizioni.

```
boolean maggiorenne = true;
boolean haSconto = false;

System.out.println(maggiorenne);
System.out.println(haSconto);
```

Output:

```
true
false
```

Un singolo carattere: char

`char` contiene un solo carattere e usa gli apici singoli.

```
char iniziale = 'L';
char voto = 'A';
```

Non confonderlo con `String`, che usa le virgolette doppie e contiene testo anche lungo.

```
char lettera = 'A';
String parola = "A";
```

Sembrano simili, ma sono tipi diversi.

Testo: String

`String` contiene testo.

```
String nome = "Luca";  
String messaggio = "Benvenuto";
```

Puoi unire testi con `+` :

```
String nome = "Luca";  
System.out.println("Ciao, " + nome);
```

Output:

```
Ciao, Luca
```

Tipi primitivi e oggetti

Java ha tipi **primitivi**, come:

- `int`
- `double`
- `boolean`
- `char`

Sono tipi semplici, pensati per valori base.

`String` , invece, è un **oggetto**. Per ora ti basta sapere che `String` offre anche metodi utili:

```
String nome = "Luca";  
System.out.println(nome.length());
```

Output:

```
4
```

`length()` conta i caratteri del testo.

Esempio completo

```
public class Tipi {  
    public static void main(String[] args) {  
        String prodotto = "Quaderno";  
        int quantita = 3;  
        double prezzo = 2.50;  
        boolean disponibile = true;  
  
        double totale = quantita * prezzo;  
  
        System.out.println("Prodotto: " + prodotto);  
        System.out.println("Quantita: " + quantita);  
        System.out.println("Totale: " + totale);  
        System.out.println("Disponibile: " + disponibile);  
    }  
}
```

Output:

```
Prodotto: Quaderno  
Quantita: 3  
Totale: 7.5  
Disponibile: true
```

Scegliere il tipo giusto rende il programma piu chiaro e aiuta Java a controllare gli errori.

Primo programma

Come scrivere, compilare ed eseguire un primo programma Java.

Il programma piu piccolo

Il primo programma serve a controllare che tutto funzioni e a vedere il percorso completo: scrivere, compilare, eseguire.

Crea un file chiamato `Ciao.java` :

```
public class Ciao {  
    public static void main(String[] args) {  
        System.out.println("Ciao!");  
    }  
}
```

Il nome del file deve essere `Ciao.java` perche la classe si chiama `Ciao` .

Attenzione: in Java maiuscole e minuscole contano. `Ciao` e `ciao` sono nomi diversi.

Compilare il programma

Nel terminale, entra nella cartella dove si trova `Ciao.java` e scrivi:

```
javac Ciao.java
```

Se non vedi messaggi, e una buona notizia: significa che la compilazione e riuscita.

Java crea un nuovo file:

```
Ciao.class
```

Questo file non lo devi modificare a mano. E il risultato della compilazione.

Eseguire il programma

Ora puoi eseguire il programma:

```
java Ciao
```

Output:

```
Ciao!
```

Quando usi `java Ciao`, stai dicendo alla JVM: "esegui la classe `Ciao`".

Capire le righe principali

Vediamo il codice con calma.

```
public class Ciao {
```

Questa riga dichiara una **classe** chiamata `Ciao`. Per ora puoi pensare alla classe come al contenitore principale del programma.

```
public static void main(String[] args) {
```

Questa è la riga del metodo `main`. Java parte da qui quando esegui il programma.

```
System.out.println("Ciao!");
```

Questa riga stampa un testo nel terminale.

Le parentesi graffe `{` e `}` indicano dove inizia e finisce un blocco di codice.

Modificare il messaggio

Prova a cambiare il testo:

```
public class Ciao {  
    public static void main(String[] args) {  
        System.out.println("Sto imparando Java.");  
    }  
}
```

Ricompila:

```
javac Ciao.java
```

Poi esegui:

```
java Ciao
```

Output:

```
Sto imparando Java.
```

Ogni volta che modifichi il file `.java`, devi ricompilare prima di eseguire la nuova versione.

Errori comuni

Se dimentichi il punto e virgola:

```
System.out.println("Ciao!")
```

Java segnala un errore di compilazione. La versione corretta è:

```
System.out.println("Ciao!");
```

Se il nome del file non corrisponde alla classe pubblica, Java segnala un altro errore. Con `public class Ciao`, il file deve chiamarsi `Ciao.java`.

Installazione

Come installare Java, configurare il JDK e preparare l'ambiente di lavoro.

Cosa devi installare

Per programmare in Java ti serve il **JDK**, cioè il Java Development Kit.

Il JDK contiene gli strumenti per:

- compilare codice Java
- eseguire programmi Java
- usare le librerie standard del linguaggio

Potresti sentire anche la parola **JRE**, Java Runtime Environment. Il JRE serve solo a eseguire programmi Java già pronti. Per scrivere codice, invece, ti serve il JDK.

Nota: se stai imparando Java, scegli sempre il JDK. E il pacchetto completo.

Installare il JDK

Puoi usare un JDK moderno, per esempio una versione LTS, cioè con supporto a lungo termine.

Le distribuzioni più comuni sono:

- Eclipse Temurin
- Oracle JDK
- Microsoft Build of OpenJDK
- Amazon Corretto

Per iniziare, una qualunque di queste va bene. L'importante è installare un JDK, non solo un JRE.

Controllare l'installazione

Dopo l'installazione, apri il terminale.

Su Windows puoi usare Prompt dei comandi o PowerShell. Su macOS e Linux puoi usare Terminale.

Scrivi:

```
java --version
```

Se Java è installato, vedrai un numero di versione.

Poi controlla anche il compilatore:

```
javac --version
```

`java` esegue i programmi. `javac` li compila.

Se `java` funziona ma `javac` no, probabilmente hai installato un runtime e non un JDK, oppure il percorso degli strumenti non è configurato correttamente.

Scegliere un editor

Puoi scrivere Java con molti editor. Per iniziare, Visual Studio Code va bene perché è leggero e gratuito.

Installa:

1. Visual Studio Code
2. L'estensione "Extension Pack for Java"

Puoi usare anche IntelliJ IDEA Community Edition. È molto usato per Java e offre molti aiuti automatici, ma all'inizio può sembrare più ricco del necessario.

Creare una cartella di lavoro

Crea una cartella per gli esercizi, per esempio:

```
java-esercizi
```

Dentro questa cartella metterai i file `.java`.

Un file Java contiene codice sorgente, cioè il testo che scrivi tu. Quando lo compili, Java crea un file `.class`, che contiene il codice pronto per la JVM.

Primo controllo completo

Crea un file chiamato `Saluto.java` con questo contenuto:

```
public class Saluto {  
    public static void main(String[] args) {  
        System.out.println("Java funziona!");  
    }  
}
```

Nel terminale, entra nella cartella dove hai salvato il file e compila:

```
javac Saluto.java
```

Poi esegui:

```
java Saluto
```

Output:

```
Java funziona!
```

Attenzione: quando esegui il programma scrivi `java Saluto`, senza `.java` e senza `.class`.

Se vedi il messaggio, l'ambiente è pronto.

Operatori

Come combinare valori con operatori aritmetici, di confronto e logici.

Cosa sono gli operatori

Gli operatori sono simboli che fanno qualcosa con i valori.

Hai già visto `=` :

```
int eta = 20;
```

Qui `=` assegna il valore `20` alla variabile `eta` .

Java ha operatori per calcolare, confrontare e combinare condizioni.

Operatori aritmetici

Gli operatori aritmetici lavorano con i numeri.

```
int a = 10;  
int b = 3;  
  
System.out.println(a + b);  
System.out.println(a - b);  
System.out.println(a * b);  
System.out.println(a / b);  
System.out.println(a % b);
```

Output:

```
13  
7  
30  
3  
1
```

Il simbolo `%` si chiama **modulo** e restituisce il resto della divisione.

`10 % 3` vale `1`, perché 10 diviso 3 fa 3 con resto 1.

Divisione tra interi

Con due `int`, Java fa una divisione intera.

```
int risultato = 10 / 3;  
System.out.println(risultato);
```

Output:

```
3
```

La parte decimale viene tagliata.

Se vuoi un risultato decimale, usa almeno un `double` :

```
double risultato = 10.0 / 3;  
System.out.println(risultato);
```

Assegnazione

`=` assegna un valore.

```
int punti = 10;  
punti = punti + 5;
```

Java offre anche forme più brevi:

```
punti += 5; // come punti = punti + 5
punti -= 2; // come punti = punti - 2
```

Per aumentare o diminuire di 1:

```
punti++;
punti--;
```

All'inizio usa pure la forma lunga. E più esplicita.

Operatori di confronto

Gli operatori di confronto producono un valore `boolean`: `true` oppure `false`.

```
int eta = 20;

System.out.println(eta > 18);
System.out.println(eta == 18);
System.out.println(eta != 18);
```

Output:

```
true
false
true
```

I principali sono:

- `>` maggiore di
- `<` minore di
- `>=` maggiore o uguale
- `<=` minore o uguale
- `==` uguale

- `!=` diverso

Attenzione: `=` assegna. `==` confronta.

Operatori logici

Gli operatori logici combinano condizioni.

```
int eta = 20;
boolean haBiglietto = true;

boolean puoEntrare = eta >= 18 && haBiglietto;
System.out.println(puoEntrare);
```

Output:

```
true
```

I principali operatori logici sono:

- `&&` significa "e": entrambe le condizioni devono essere vere
- `||` significa "oppure": basta una condizione vera
- `!` significa "non": inverte il valore

```
boolean piove = false;
System.out.println(!piove);
```

Output:

```
true
```

Precedenza

Java esegue alcune operazioni prima di altre.

```
int risultato = 2 + 3 * 4;  
System.out.println(risultato);
```

Output:

14

La moltiplicazione avviene prima dell'addizione.

Se vuoi rendere l'ordine chiaro, usa le parentesi:

```
int risultato = (2 + 3) * 4;  
System.out.println(risultato);
```

Output:

20

Le parentesi non servono solo al computer. Servono anche a chi legge il codice.

Struttura di un programma

Come sono organizzati classi, metodi e istruzioni in un file Java.

Guardare un programma a strati

Un programma Java sembra lungo anche quando fa poco. Questo succede perché Java vuole una struttura esplicita.

Partiamo da un esempio:

```
public class Programma {  
    public static void main(String[] args) {  
        System.out.println("Inizio");  
        System.out.println("Fine");  
    }  
}
```

Puoi leggerlo a strati:

- la classe contiene il programma
- il metodo `main` contiene le istruzioni da eseguire
- le istruzioni sono le singole azioni

La classe

```
public class Programma {  
    ...  
}
```

Una **classe** è un blocco che raccoglie codice. Nei primi esempi useremo una classe come contenitore principale.

Il nome della classe pubblica deve corrispondere al nome del file.

Se scrivi:

```
public class Programma {  
}
```

il file deve chiamarsi:

```
Programma.java
```


Il metodo main

```
public static void main(String[] args) {  
    ...  
}
```

Il metodo `main` è il punto di partenza del programma. Quando scrivi `java Programma`, Java cerca questo metodo e inizia da lì.

Per ora non serve capire ogni parola della riga. Tienila come formula di avvio:

```
public static void main(String[] args)
```

La capirai meglio quando parleremo di metodi, classi e oggetti.

Le istruzioni

Dentro `main` scriviamo le istruzioni:

```
System.out.println("Inizio");  
System.out.println("Fine");
```

Una **istruzione** è un comando che Java deve eseguire.

In Java molte istruzioni finiscono con il punto e virgola `;`.

```
int eta = 20;  
System.out.println(eta);
```

Il punto e virgola dice a Java: "questa istruzione è finita".

Le parentesi graffe

Le parentesi graffe delimitano un blocco:

```
{  
    // codice dentro il blocco  
}
```

Nel nostro programma ci sono due blocchi:

```
public class Programma {  
    public static void main(String[] args) {  
        System.out.println("Ciao");  
    }  
}
```

Il blocco di `main` sta dentro il blocco della classe.

L'indentazione, cioè gli spazi all'inizio delle righe, non cambia il significato per Java. Serve però a noi per leggere meglio.

Un programma con più istruzioni

```
public class Scontrino {  
    public static void main(String[] args) {  
        double prezzo = 12.50;  
        double spedizione = 4.90;  
        double totale = prezzo + spedizione;  
  
        System.out.println("Prezzo: " + prezzo);  
        System.out.println("Spedizione: " + spedizione);  
        System.out.println("Totale: " + totale);  
    }  
}
```

Output:

```
Prezzo: 12.5  
Spedizione: 4.9  
Totale: 17.4
```

Il programma viene eseguito dall'alto verso il basso, una riga alla volta.

Regola pratica

Quando scrivi un file Java di base, controlla tre cose:

1. il nome del file corrisponde al nome della classe
2. il metodo `main` è scritto correttamente
3. ogni istruzione che lo richiede finisce con `;`

Molti errori iniziali nascono da uno di questi tre dettagli.

Stringhe

Come lavorare con testi, concatenazione e metodi comuni di String.

Cosa è una stringa

Una stringa è un testo. In Java il tipo per i testi è `String`.

```
String nome = "Luca";  
String messaggio = "Benvenuto";
```

Le stringhe usano le virgolette doppie.

```
String parola = "ciao";
```

Gli apici singoli, invece, servono per un solo carattere:

```
char iniziale = 'L';
```

Concatenare stringhe

Per unire stringhe, usa `+`.

```
String nome = "Luca";  
String saluto = "Ciao, " + nome + "!";  
  
System.out.println(saluto);
```

Output:

```
Ciao, Luca!
```

Puoi unire anche numeri:

```
int eta = 20;  
System.out.println("Eta: " + eta);
```

Java converte il numero in testo per costruire il messaggio.

Contare i caratteri con length

Le stringhe hanno metodi, cioè azioni che puoi chiamare sul valore.

`length()` conta i caratteri:

```
String nome = "Sara";  
System.out.println(nome.length());
```

Output:

```
4
```

La chiamata si legge così: "sulla stringa `nome` , esegui il metodo `length` ".

Confrontare stringhe con equals

Per confrontare il contenuto di due stringhe, usa `equals` .

```
String password = "segreto";

System.out.println(password.equals("segreto"));
System.out.println(password.equals("ciao"));
```

Output:

```
true
false
```

Attenzione: con le stringhe evita `==` per confrontare il testo. Usa `equals` .

`==` controlla se due riferimenti puntano allo stesso oggetto. `equals` controlla se il contenuto è uguale.

Per iniziare, ricordati questa regola pratica:

```
testo.equals("valore")
```

Prendere una parte con substring

`substring` prende una parte del testo.

```
String parola = "programmare";  
String parte = parola.substring(0, 7);  
  
System.out.println(parte);
```

Output:

```
program
```

Gli indici partono da **0** . Il primo numero è incluso, il secondo è escluso.

In **"programmare"** :

- indice **0** : p
- indice **1** : r
- indice **2** : o

substring(0, 7) prende i caratteri dagli indici 0 a 6.

Cambiare maiuscole e minuscole

Altri metodi utili:

```
String nome = "Luca";  
  
System.out.println(nome.toUpperCase());  
System.out.println(nome.toLowerCase());
```

Output:

```
LUCA  
luca
```

Questi metodi non modificano la stringa originale. Creano un nuovo testo.

Esempio completo

```
public class NomeUtente {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Nome: ");  
        String nome = scanner.nextLine();  
  
        System.out.println("Ciao, " + nome + "!");  
        System.out.println("Il tuo nome ha " + nome.length() + "  
caratteri.");  
        System.out.println("In maiuscolo: " + nome.toUpperCase());  
  
        scanner.close();  
    }  
}
```

Le stringhe saranno ovunque nei programmi Java: messaggi, input, nomi, percorsi di file e dati letti dall'esterno.

Conversione dei tipi

Come convertire valori tra tipi diversi ed evitare errori comuni.

Perche convertire un tipo

A volte hai un valore in un tipo, ma ti serve in un altro.

Per esempio:

- leggi un numero come testo e vuoi usarlo per un calcolo
- hai un `int` e vuoi salvarlo in un `double`
- hai un `double` e vuoi prendere solo la parte intera

Questa operazione si chiama **conversione di tipo**.

Conversioni automatiche

Java permette alcune conversioni senza istruzioni speciali.

Per esempio, un `int` può diventare un `double` :

```
int eta = 20;
double etaDecimale = eta;

System.out.println(etaDecimale);
```

Output:

```
20.0
```

Questa conversione è sicura: un numero intero può essere rappresentato come numero decimale senza perdere informazione.

Cast esplicito

La conversione opposta non è automatica.

```
double prezzo = 12.90;
int prezzoIntero = prezzo; // errore
```

Java non vuole perdere la parte decimale senza che tu lo dica chiaramente.

Per farlo usi un **cast**:

```
double prezzo = 12.90;
int prezzoIntero = (int) prezzo;

System.out.println(prezzoIntero);
```

Output:

12

Il cast a `int` taglia la parte decimale. Non arrotonda.

Perdita di informazioni

Questo dettaglio è importante:

```
double media = 7.8;
int voto = (int) media;

System.out.println(voto);
```

Output:

7

La parte `.8` sparisce.

Se vuoi arrotondare, usa un metodo apposito:

```
double media = 7.8;
long arrotondato = Math.round(media);

System.out.println(arrotondato);
```

Output:

8

Convertire testo in numero

Quando leggi input dall'utente o da un file, spesso i dati arrivano come testo.

Per trasformare una `String` in un `int`, usa `Integer.parseInt`:

```
String testo = "25";  
int numero = Integer.parseInt(testo);  
  
System.out.println(numero + 5);
```

Output:

```
30
```

Per un `double`, usa `Double.parseDouble`:

```
String testo = "12.50";  
double prezzo = Double.parseDouble(testo);  
  
System.out.println(prezzo * 2);
```

Output:

```
25.0
```

Errore comune: testo non valido

Se il testo non contiene un numero valido, Java segnala un errore.

```
String testo = "ciao";  
int numero = Integer.parseInt(testo); // errore a runtime
```

"ciao" non può diventare un numero intero.

Più avanti vedremo come gestire questi casi con le eccezioni.

Convertire numero in testo

Per unire un numero a una stringa, spesso basta `+` :

```
int eta = 20;  
String messaggio = "Eta: " + eta;  
  
System.out.println(messaggio);
```

Output:

```
Eta: 20
```

Java converte il numero in testo per poter costruire la frase.

Regola pratica

Chiediti sempre:

- sto passando da un tipo più piccolo a uno più ampio?
- rischio di perdere dati?
- il testo contiene davvero un numero?

Se la conversione può perdere informazione, Java ti chiede di essere esplicito. È una protezione, non un fastidio.

Variabili

Come salvare valori in Java, dichiarare variabili e modificarle nel tempo.

Cos'è una variabile

Un programma deve ricordare informazioni mentre lavora: un nome, un prezzo, un punteggio, un totale.

Una **variabile** è una specie di scatola con un'etichetta. L'etichetta è il nome della variabile. Il contenuto è il valore salvato.

```
String nome = "Luca";  
int eta = 20;
```

Qui creiamo due variabili:

- `nome` contiene il testo `"Luca"`
- `eta` contiene il numero `20`

Dichiarare una variabile

In Java devi indicare il tipo della variabile prima del nome.

```
int eta;
```

Questa riga dice: "crea una variabile chiamata `eta` che conterrà numeri interi".

Puoi anche darle subito un valore:

```
int eta = 20;
```

Questa operazione si chiama **inizializzazione**.

Leggere il valore

Per usare il valore, scrivi il nome della variabile.

```
public class Variabili {  
    public static void main(String[] args) {  
        String nome = "Luca";  
        int eta = 20;  
  
        System.out.println(nome);  
        System.out.println(eta);  
    }  
}
```

Output:

```
Luca  
20
```

Quando Java vede `nome`, prende il valore salvato dentro quella variabile.

Modificare una variabile

Puoi cambiare il valore di una variabile con `=`.

```
int eta = 20;  
System.out.println(eta);  
  
eta = 21;  
System.out.println(eta);
```

Output:

```
20  
21
```

La seconda assegnazione sostituisce il valore precedente.

Puoi anche usare il valore attuale per calcolarne uno nuovo:

```
int punti = 10;  
punti = punti + 5;  
  
System.out.println(punti);
```

Output:

```
15
```

Il tipo conta

Java controlla che il valore sia compatibile con il tipo.

```
int eta = 20;  
eta = "venti"; // errore
```

`int` contiene numeri interi. Non può contenere testo.

Questo controllo può sembrare rigido, ma aiuta a trovare molti errori prima di eseguire il programma.

Nomi validi

I nomi delle variabili devono seguire alcune regole:

- possono contenere lettere, numeri e underscore `_`
- non possono iniziare con un numero
- non possono contenere spazi

- non possono essere parole riservate come `class`, `int`, `if`
- maiuscole e minuscole contano

Esempi validi:

```
String nome;  
int etaUtente;  
double prezzoTotale;
```

Esempi non validi:

```
int 2eta;           // inizia con un numero  
String nome utente; // contiene uno spazio  
int class;          // parola riservata
```

Convenzione: camelCase

In Java i nomi delle variabili usano spesso il **camelCase**.

```
int etaMinima = 18;  
double prezzoTotale = 24.90;  
String nomeUtente = "Sara";
```

La prima parola inizia minuscola. Le parole successive iniziano con la maiuscola.

Esempio completo

```
public class Profilo {  
    public static void main(String[] args) {  
        String nome = "Sara";  
        int eta = 17;  
        boolean studente = true;  
  
        System.out.println("Nome: " + nome);  
        System.out.println("Eta: " + eta);  
        System.out.println("Studente: " + studente);  
  
        eta = eta + 1;  
        System.out.println("Eta aggiornata: " + eta);  
    }  
}
```

Output:

```
Nome: Sara  
Eta: 17  
Studente: true  
Eta aggiornata: 18
```

Qui `eta` cambia, mentre `nome` e `studente` restano uguali.

ArrayList

Come usare liste dinamiche quando il numero di elementi puo cambiare.

Perche usare ArrayList

Un array ha una dimensione fissa. Se lo crei con 3 elementi, resta di 3 elementi.

`ArrayList` è una lista dinamica: puoi aggiungere e rimuovere elementi mentre il programma gira.

Prima devi importarla:

Creare una ArrayList

```
public class ListaNomi {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        System.out.println(nomi);  
    }  
}
```

Output:

```
[Luca, Sara, Mina]
```

`ArrayList` significa: "lista di stringhe".

Aggiungere elementi

Usa `add` :

```
ArrayList prodotti = new ArrayList<>();  
  
prodotti.add("Pane");  
prodotti.add("Latte");  
prodotti.add("Mele");
```

Ogni `add` aggiunge un elemento in fondo alla lista.

Leggere elementi

Usa `get` con l'indice.

```
ArrayList nomi = new ArrayList<>();  
nomi.add("Luca");  
nomi.add("Sara");  
  
System.out.println(nomi.get(0));
```

Output:

```
Luca
```

Anche qui gli indici partono da zero.

Modificare elementi

Usa `set` .

```
ArrayList nomi = new ArrayList<>();  
nomi.add("Luca");  
nomi.add("Sara");  
  
nomi.set(1, "Giulia");  
  
System.out.println(nomi);
```

Output:

```
[Luca, Giulia]
```

Rimuovere elementi

Usa `remove` .

```
ArrayList prodotti = new ArrayList<>();
prodotti.add("Pane");
prodotti.add("Latte");
prodotti.add("Mele");

prodotti.remove("Latte");

System.out.println(prodotti);
```

Output:

```
[Pane, Mele]
```

Puoi rimuovere anche per indice:

```
prodotti.remove(0);
```

size

`size()` restituisce il numero di elementi.

```
ArrayList nomi = new ArrayList<>();
nomi.add("Luca");
nomi.add("Sara");

System.out.println(nomi.size());
```

Output:

2

Negli array usi `length` . Nelle `ArrayList` usi `size()` .

Array o ArrayList?

Usa un array quando:

- la dimensione è fissa
- vuoi una struttura semplice
- stai facendo esercizi sugli indici

Usa `ArrayList` quando:

- il numero di elementi può cambiare
- devi aggiungere o rimuovere valori
- vuoi una lista più comoda da gestire

Esempio completo

```
public class Spesa {
    public static void main(String[] args) {
        ArrayList lista = new ArrayList<>();

        lista.add("Pane");
        lista.add("Latte");
        lista.add("Mele");

        lista.remove("Latte");
        lista.add("Pasta");

        for (int i = 0; i < lista.size(); i++) {
            System.out.println("- " + lista.get(i));
        }
    }
}
```

Output:

- Pane
- Mele
- Pasta

`ArrayList` è una delle strutture più usate in Java quando lavori con gruppi di dati.

Array

Come raccogliere più valori dello stesso tipo in una struttura ordinata.

Cosa è un array

Un array raccoglie più valori dello stesso tipo in una sequenza ordinata.

Pensalo come una fila di caselle numerate.

```
int[] voti = {7, 8, 6, 9};
```

Qui `voti` contiene quattro numeri.

Gli indici partono da zero

Per leggere un elemento, usi il suo indice.

```
int[] voti = {7, 8, 6, 9};  
  
System.out.println(voti[0]);  
System.out.println(voti[1]);
```

Output:

7
8

Il primo elemento ha indice `0` , non `1` .

Modificare un elemento

Puoi cambiare il valore di una casella:

```
int[] voti = {7, 8, 6, 9};  
  
voti[2] = 10;  
  
System.out.println(voti[2]);
```

Output:

10

Prima `voti[2]` valeva `6` . Dopo l'assegnazione vale `10` .

Creare un array vuoto

Puoi creare un array indicando la sua dimensione:

```
String[] nomi = new String[3];  
  
nomi[0] = "Luca";  
nomi[1] = "Sara";  
nomi[2] = "Mina";
```

La dimensione di un array non cambia dopo la creazione.

Se crei un array di 3 elementi, avrà sempre 3 caselle.

length

`length` contiene il numero di elementi.

```
String[] nomi = {"Luca", "Sara", "Mina"};

System.out.println(nomi.length);
```

Output:

```
3
```

Negli array `length` non ha parentesi. Non scrivere `length()` .

Attraversare un array con for

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(nomi[i]);
}
```

Output:

```
Luca
Sara
Mina
```

Questa forma è utile quando ti serve l'indice.

Errore comune: indice fuori limite

```
int[] numeri = {10, 20, 30};  
System.out.println(numeri[3]); // errore
```

Gli indici validi sono:

- 0
- 1
- 2

3 è fuori dall'array.

Java segnala un errore chiamato `ArrayIndexOutOfBoundsException`.

Esempio: media dei voti

```
public class Media {  
    public static void main(String[] args) {  
        int[] voti = {7, 8, 6, 9};  
        int somma = 0;  
  
        for (int i = 0; i < voti.length; i++) {  
            somma = somma + voti[i];  
        }  
  
        double media = (double) somma / voti.length;  
        System.out.println("Media: " + media);  
    }  
}
```

Output:

```
Media: 7.5
```

L'array è utile quando hai molti valori simili e vuoi trattarli insieme.

HashMap

Come associare chiavi e valori per rappresentare dati cercabili.

Cosa è una HashMap

Una `HashMap` collega una **chiave** a un **valore**.

Pensala come un piccolo dizionario:

- chiave: parola
- valore: significato

Oppure come una rubrica:

- chiave: nome
- valore: numero di telefono

Prima devi importarla:

Creare una HashMap

```
public class Rubrica {  
    public static void main(String[] args) {  
        HashMap telefoni = new HashMap<>();  
  
        telefoni.put("Luca", "3331234567");  
        telefoni.put("Sara", "3337654321");  
  
        System.out.println(telefoni);  
    }  
}
```

`HashMap` significa: chiavi di tipo `String`, valori di tipo `String`.

Aggiungere con put

Usa `put` per aggiungere una coppia chiave-valore.

```
HashMap eta = new HashMap<>();  
  
eta.put("Luca", 20);  
eta.put("Sara", 25);
```

Se usi una chiave già presente, il valore viene sostituito.

```
eta.put("Luca", 21);
```

Ora `"Luca"` vale `21` .

Leggere con get

Usa `get` per recuperare il valore associato a una chiave.

```
HashMap eta = new HashMap<>();  
eta.put("Luca", 20);  
  
System.out.println(eta.get("Luca"));
```

Output:

```
20
```

Se la chiave non esiste, `get` restituisce `null` .

Controllare se una chiave esiste

Prima di leggere, puoi usare `containsKey` .

```
HashMap eta = new HashMap<>();
eta.put("Luca", 20);

if (eta.containsKey("Mina")) {
    System.out.println(eta.get("Mina"));
} else {
    System.out.println("Mina non e presente.");
}
```

Output:

```
Mina non e presente.
```

Rimuovere una chiave

Usa `remove` .

```
HashMap eta = new HashMap<>();
eta.put("Luca", 20);
eta.put("Sara", 25);

eta.remove("Luca");

System.out.println(eta);
```

Output:

```
{Sara=25}
```

Attraversare le chiavi

Puoi leggere tutte le chiavi con `keySet()` .

```
HashMap eta = new HashMap<>();
eta.put("Luca", 20);
eta.put("Sara", 25);

for (String nome : eta.keySet()) {
    System.out.println(nome + ": " + eta.get(nome));
}
```

Output possibile:

```
Luca: 20
Sara: 25
```

L'ordine non e garantito. Una `HashMap` serve a cercare per chiave, non a mantenere un ordine preciso.

Esempio: prezzi dei prodotti

```
public class Prezzi {  
    public static void main(String[] args) {  
        HashMap prezzi = new HashMap<>();  
  
        prezzi.put("pane", 1.50);  
        prezzi.put("latte", 1.20);  
        prezzi.put("pasta", 0.90);  
  
        String prodotto = "latte";  
  
        if (prezzi.containsKey(prodotto)) {  
            System.out.println("Prezzo: " + prezzi.get(prodotto));  
        } else {  
            System.out.println("Prodotto non trovato.");  
        }  
    }  
}
```

HashMap è utile quando vuoi cercare rapidamente un valore partendo da una chiave.

Set

Come conservare elementi unici senza duplicati.

Cosa è un Set

Un **Set** è una collezione che contiene elementi unici.

Se provi ad aggiungere due volte lo stesso valore, il duplicato non viene inserito.

Per iniziare useremo **HashSet**.

Creare un HashSet

```
public class NomiUnici {  
    public static void main(String[] args) {  
        HashSet nomi = new HashSet<>();  
  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Luca");  
  
        System.out.println(nomi);  
    }  
}
```

Output possibile:

```
[Luca, Sara]
```

"Luca" compare una sola volta.

Aggiungere elementi

Usa `add` .

```
HashSet parole = new HashSet<>();  
  
parole.add("java");  
parole.add("codice");  
parole.add("java");
```

Il secondo `"java"` viene ignorato perché esiste già.

Controllare se un elemento esiste

Usa `contains` .

```
HashSet invitati = new HashSet<>();
invitati.add("Luca");
invitati.add("Sara");

if (invitati.contains("Sara")) {
    System.out.println("Sara e nella lista.");
}
```

Output:

```
Sara e nella lista.
```

Rimuovere elementi

Usa `remove` .

```
HashSet invitati = new HashSet<>();
invitati.add("Luca");
invitati.add("Sara");

invitati.remove("Luca");

System.out.println(invitati);
```

Output:

```
[Sara]
```

Attraversare un Set

Puoi usare il ciclo for-each:

```
HashSet linguaggi = new HashSet<>();
linguaggi.add("Java");
linguaggi.add("Python");
linguaggi.add("PHP");

for (String linguaggio : linguaggi) {
    System.out.println(linguaggio);
}
```

L'ordine non è garantito. Un `HashSet` pensa all'unicità, non alla posizione.

Set, ArrayList o array?

Usa un array quando hai una dimensione fissa.

Usa `ArrayList` quando vuoi una lista ordinata che può crescere.

Usa `HashSet` quando ti interessa evitare duplicati e controllare rapidamente se un elemento esiste.

Esempio: parole uniche

```
public class ParoleUniche {
    public static void main(String[] args) {
        String[] parole = {"java", "codice", "java", "metodo", "codice"};
        HashSet uniche = new HashSet<>();

        for (int i = 0; i < parole.length; i++) {
            uniche.add(parole[i]);
        }

        System.out.println(uniche);
    }
}
```


Output possibile:

```
[java, metodo, codice]
```

Il risultato contiene ogni parola una sola volta.

Errori comuni

Gli errori Java più frequenti quando si inizia e come leggerli.

Gli errori fanno parte del lavoro

Quando impari Java, vedrai molti messaggi di errore. Non significano che stai sbagliando percorso.

Un errore è un'informazione: Java ti sta dicendo che qualcosa non torna.

La cosa importante è imparare a leggere il messaggio con calma.

Errori di compilazione

Un errore di compilazione succede quando `javac` non riesce a trasformare il file `.java` in `.class`.

Esempio:

```
public class Errore {  
    public static void main(String[] args) {  
        System.out.println("Ciao")  
    }  
}
```

Manca il punto e virgola.

Java potrebbe mostrare un messaggio simile:

```
error: ';' expected
```

Correzione:

```
System.out.println("Ciao");
```

Nome della classe e nome del file

Se scrivi:

```
public class Saluto {  
}
```

il file deve chiamarsi:

```
Saluto.java
```

Se il file ha un altro nome, Java segnala errore.

Variabile non trovata

```
public class Esempio {  
    public static void main(String[] args) {  
        int eta = 20;  
        System.out.println(anni);  
    }  
}
```

`anni` non esiste.

Java segnala che non trova quel simbolo.

Correzione:

```
System.out.println(eta);
```

Controlla spesso nomi scritti in modo diverso: `eta` , `Eta` e `età` non sono la stessa cosa.

Tipi incompatibili

```
int eta = "venti";
```

`int` richiede un numero intero. `"venti"` è una stringa.

Correzione:

```
int eta = 20;
```

Oppure:

```
String eta = "venti";
```

Dipende da cosa ti serve davvero.

Errori a runtime

Un errore a runtime succede mentre il programma è già in esecuzione.

Esempio:

```
int[] numeri = {10, 20, 30};  
System.out.println(numeri[3]);
```

Il codice compila, ma durante l'esecuzione Java segnala:

```
ArrayIndexOutOfBoundsException
```

L'array ha indici `0`, `1`, `2`. L'indice `3` non esiste.

Leggere lo stack trace

Quando un programma fallisce, Java mostra spesso uno **stack trace**.

Sembra lungo, ma all'inizio cerca soprattutto:

- il nome dell'errore
- il file
- il numero di riga

Esempio:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException  
    at Esempio.main(Esempio.java:4)
```

La parte utile è:

```
Esempio.java:4
```

Vai alla riga 4 e controlla cosa succede lì.

Metodo di controllo

Quando trovi un errore:

1. leggi la prima riga del messaggio
2. cerca il numero di riga
3. controlla nomi, parentesi e punto e virgola
4. verifica i tipi delle variabili
5. se serve, stampa valori intermedi

Gli errori diventano meno spaventosi quando li tratti come indizi.

Debugging

Come trovare problemi nel codice usando stampe, debugger e ragionamento.

Cosa significa fare debugging

Fare debugging significa cercare e correggere un problema nel codice.

Non è solo "togliere errori". È capire cosa il programma sta facendo davvero, passo dopo passo.

Leggere prima il messaggio

Quando Java mostra un errore, non partire subito a cambiare codice a caso.

Prima cerca:

- tipo di errore
- file
- numero di riga
- valore o nome citato nel messaggio

Esempio:

```
Exception in thread "main" java.lang.NumberFormatException  
    at Esempio.main(Esempio.java:8)
```

Vai alla riga 8 e chiediti: sto convertendo testo in numero? Il testo contiene davvero un numero?

Usare stampe temporanee

Un modo semplice per capire cosa succede e stampare valori.

```
int prezzo = 10;
int quantita = 3;
int totale = prezzo * quantita;

System.out.println("prezzo = " + prezzo);
System.out.println("quantita = " + quantita);
System.out.println("totale = " + totale);
```

Queste stampe sono temporanee. Servono durante il controllo.

Quando hai trovato il problema, rimuovile o trasformale in output utile.

Isolare il problema

Se un programma è lungo, prova a restringere il punto del problema.

Chiediti:

1. il valore entra corretto?
2. cambia nel punto giusto?
3. viene passato al metodo corretto?
4. l'errore compare prima o dopo questa riga?

Puoi commentare temporaneamente alcune parti o creare un esempio più piccolo.

Usare il debugger

Molti editor, come Visual Studio Code o IntelliJ IDEA, hanno un debugger.

Il debugger permette di:

- fermare il programma su una riga
- eseguire una riga alla volta
- guardare il valore delle variabili
- capire quale ramo di un `if` viene eseguito

Il punto in cui fermi il programma si chiama **breakpoint**.

Un esempio pratico

Immagina questo codice:

```
int[] voti = {7, 8, 6};
int somma = 0;

for (int i = 0; i <= voti.length; i++) {
    somma = somma + voti[i];
}

System.out.println(somma);
```

Il programma fallisce. Il problema e nella condizione:

```
i <= voti.length
```

L'ultimo indice valido e `voti.length - 1`.

Correzione:

```
for (int i = 0; i < voti.length; i++) {
    somma = somma + voti[i];
}
```

Metodo semplice

Quando non capisci un errore:

1. riprodurlo
2. leggi il messaggio
3. trova la riga
4. stampa o osserva le variabili vicine
5. fai una modifica piccola
6. ricontrolla

Il debugging richiede pazienza. Una modifica piccola alla volta ti evita di creare nuovi problemi mentre cerchi quello iniziale.

Eccezioni

Come gestire situazioni impreviste con try, catch e finally.

Cosa e un'eccezione

Un'eccezione e un problema che avviene mentre il programma e in esecuzione.

Esempi:

- un file non esiste
- l'utente inserisce testo al posto di un numero
- un array viene letto fuori dai suoi limiti

Java usa le eccezioni per segnalare questi casi.

try e catch

Con `try catch` puoi provare un'operazione e gestire l'errore se succede.

```
try {  
    int numero = Integer.parseInt("abc");  
    System.out.println(numero);  
} catch (NumberFormatException e) {  
    System.out.println("Non e un numero valido.");  
}
```

Output:

```
Non e un numero valido.
```

`Integer.parseInt("abc")` fallisce perche `"abc"` non e un numero.

Un esempio con input

```
public class LetturaNumero {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Numero: ");
        String testo = scanner.nextLine();

        try {
            int numero = Integer.parseInt(testo);
            System.out.println("Doppio: " + (numero * 2));
        } catch (NumberFormatException e) {
            System.out.println("Devi inserire un numero intero.");
        }

        scanner.close();
    }
}
```

Se l'utente scrive `12`, il programma calcola il doppio.

Se scrive `ciao`, il programma mostra un messaggio chiaro invece di interrompersi.

finally

`finally` contiene codice che viene eseguito comunque, sia se tutto va bene sia se c'è un errore.

```
try {
    System.out.println("Provo a fare qualcosa");
} catch (Exception e) {
    System.out.println("Qualcosa è andato storto");
} finally {
    System.out.println("Questa riga viene eseguita sempre");
}
```

È utile per chiudere risorse, come file o connessioni.

Eccezioni controllate e non controllate

Java distingue due famiglie importanti.

Le **eccezioni controllate** devono essere gestite o dichiarate. Un esempio comune è `IOException`, usata quando lavori con file.

Le **eccezioni non controllate** possono avvenire a runtime e Java non ti obbliga sempre a gestirle. Esempi:

- `NumberFormatException`
- `ArrayIndexOutOfBoundsException`
- `NullPointerException`

Per iniziare non devi memorizzare tutte le categorie. Ti basta sapere che alcune operazioni, come i file, obbligano a pensare agli errori.

Non catturare tutto senza motivo

Potresti scrivere:

```
catch (Exception e) {  
    System.out.println("Errore");  
}
```

Funziona, ma è molto generico.

Quando puoi, cattura l'eccezione specifica:

```
catch (NumberFormatException e) {  
    System.out.println("Numero non valido.");  
}
```

Il messaggio è più chiaro e il codice è più facile da controllare.

Regola pratica

Usa le eccezioni per gestire problemi che possono capitare anche in un programma scritto bene: input sbagliato, file mancanti, dati non validi.

Non usarle per nascondere errori che dovresti correggere nel codice.

break e continue

Come interrompere o saltare un passaggio dentro un ciclo.

Controllare meglio un ciclo

A volte un ciclo non deve arrivare alla sua fine naturale.

Java offre due parole utili:

- `break` interrompe il ciclo
- `continue` salta al giro successivo

Usale con misura: rendono alcuni casi più semplici, ma se abusate possono rendere il flusso difficile da seguire.

break

`break` esce subito dal ciclo.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 4) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3
```

Quando `i` vale `4`, Java esegue `break` e il ciclo finisce.

Cercare un valore

`break` è utile quando hai trovato quello che cercavi.

```
int[] numeri = {4, 8, 15, 16, 23, 42};
int cercato = 16;
boolean trovato = false;

for (int i = 0; i < numeri.length; i++) {
    if (numeri[i] == cercato) {
        trovato = true;
        break;
    }
}

System.out.println(trovato);
```

Output:

```
true
```

Dopo aver trovato `16`, non serve controllare gli altri numeri.

continue

`continue` salta il resto del blocco e passa al prossimo giro.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }

    System.out.println(i);
}
```

Output:

```
1  
2  
4  
5
```

Quando `i` vale `3`, Java salta la stampa e continua con `4`.

Saltare valori non utili

Esempio: stampiamo solo i numeri positivi.

```
int[] numeri = {5, -2, 8, -1, 0, 3};  
  
for (int i = 0; i < numeri.length; i++) {  
    if (numeri[i] <= 0) {  
        continue;  
    }  
  
    System.out.println(numeri[i]);  
}
```

Output:

```
5  
8  
3
```

I numeri minori o uguali a zero vengono saltati.

break in un while

Puoi usare `break` anche con `while`.

```
public class Menu {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        while (true) {  
            System.out.print("Scrivi esci per terminare: ");  
            String comando = scanner.nextLine();  
  
            if (comando.equals("esci")) {  
                break;  
            }  
  
            System.out.println("Hai scritto: " + comando);  
        }  
  
        scanner.close();  
    }  
}
```

`while (true)` crea un ciclo infinito intenzionale. `break` decide quando fermarlo.

Regola pratica

Usa `break` quando:

- hai trovato un risultato
- l'utente vuole uscire
- continuare sarebbe inutile

Usa `continue` quando:

- vuoi saltare solo un caso
- il resto del ciclo non serve per quel giro

Se il codice diventa difficile da leggere, prova prima a riscrivere la condizione del ciclo.

Condizioni

Come prendere decisioni nel codice con `if`, `else` e `switch`.

A cosa servono le condizioni

Una condizione permette al programma di scegliere cosa fare.

E come un bivio:

- se succede una cosa, fai questo
- altrimenti, fai qualcos'altro

In Java le condizioni usano valori `boolean` : `true` o `false` .

if

`if` esegue un blocco solo se la condizione è vera.

```
int eta = 20;

if (eta >= 18) {
    System.out.println("Puoi entrare.");
}
```

Output:

```
Puoi entrare.
```

La condizione è:

```
eta >= 18
```

Se vale `true` , Java esegue il blocco tra parentesi graffe.

else

`else` indica cosa fare quando la condizione è falsa.

```
int eta = 16;

if (eta >= 18) {
    System.out.println("Puoi entrare.");
} else {
    System.out.println("Non puoi entrare.");
}
```

Output:

```
Non puoi entrare.
```

else if

Quando hai piu casi, usa `else if` .

```
int voto = 7;

if (voto >= 9) {
    System.out.println("Ottimo");
} else if (voto >= 6) {
    System.out.println("Sufficiente");
} else {
    System.out.println("Da riprovare");
}
```

Output:

```
Sufficiente
```

Java controlla dall'alto verso il basso. Appena trova una condizione vera, esegue quel blocco e salta gli altri.

Condizioni con operatori logici

Puoi combinare piu condizioni.

```
int eta = 20;
boolean haBiglietto = true;

if (eta >= 18 && haBiglietto) {
    System.out.println("Ingresso consentito.");
}
```

`&&` significa "e": entrambe le condizioni devono essere vere.

Con `||`, invece, basta una condizione vera:

```
boolean haPassword = false;
boolean haCodiceTemporaneo = true;

if (haPassword || haCodiceTemporaneo) {
    System.out.println("Accesso possibile.");
}
```

switch

`switch` e comodo quando confronti una stessa variabile con piu valori precisi.

```
int giorno = 2;

switch (giorno) {
    case 1:
        System.out.println("Lunedì");
        break;
    case 2:
        System.out.println("Martedì");
        break;
    case 3:
        System.out.println("Mercoledì");
        break;
    default:
        System.out.println("Giorno non valido");
}
```

Output:

```
Martedì
```

break ferma lo **switch** dopo il caso trovato.

default viene eseguito se nessun caso corrisponde.

Quando usare if e quando switch

Usa **if** quando hai condizioni diverse:

```
if (eta >= 18 && haBiglietto) {
    ...
}
```

Usa **switch** quando controlli molti valori dello stesso dato:

```
switch (scelta) {  
    case 1:  
        ...  
}
```

Esempio completo

```
public class Menu {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("1. Saluta");  
        System.out.println("2. Mostra aiuto");  
        System.out.print("Scelta: ");  
  
        int scelta = scanner.nextInt();  
  
        switch (scelta) {  
            case 1:  
                System.out.println("Ciao!");  
                break;  
            case 2:  
                System.out.println("Scegli un numero dal menu.");  
                break;  
            default:  
                System.out.println("Scelta non valida.");  
        }  
  
        scanner.close();  
    }  
}
```

Le condizioni sono uno dei primi strumenti che trasformano un programma da sequenza fissa a programma capace di reagire.

Ciclo for-each

Come attraversare array e collezioni senza gestire manualmente gli indici.

Un modo piu semplice per attraversare dati

Quando vuoi leggere tutti gli elementi di un array o di una collezione, spesso non ti serve l'indice.

In questi casi puoi usare il ciclo for-each.

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (String nome : nomi) {
    System.out.println(nome);
}
```

Output:

```
Luca
Sara
Mina
```

Leggere la sintassi

```
for (String nome : nomi) {
    ...
}
```

Si legge così:

"per ogni `String` chiamata `nome` dentro `nomi`, esegui questo blocco".

`nome` è una variabile temporanea. A ogni giro contiene un elemento diverso.

for-each con array

```
int[] voti = {7, 8, 6, 9};

for (int voto : voti) {
    System.out.println(voto);
}
```

Output:

```
7
8
6
9
```

Il codice e' piu' breve rispetto a un `for` con indice.

for-each con ArrayList

```
public class Lista {
    public static void main(String[] args) {
        ArrayList prodotti = new ArrayList<>();
        prodotti.add("Pane");
        prodotti.add("Latte");
        prodotti.add("Mele");

        for (String prodotto : prodotti) {
            System.out.println(prodotto);
        }
    }
}
```

Output:

Pane
Latte
Mele

Quando e comodo

Il for-each e comodo quando vuoi:

- stampare tutti gli elementi
- sommare tutti i numeri
- cercare un valore
- controllare ogni elemento senza usare la posizione

Esempio:

```
int[] prezzi = {10, 20, 15};  
int totale = 0;  
  
for (int prezzo : prezzi) {  
    totale = totale + prezzo;  
}  
  
System.out.println(totale);
```

Output:

45

Limite: non hai l'indice

Con for-each non hai direttamente la posizione dell'elemento.

Se devi stampare anche l'indice, usa un `for` normale:

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(i + ": " + nomi[i]);
}
```

Output:

```
0: Luca
1: Sara
2: Mina
```

Limite: modificare elementi

Con un array di numeri, questa modifica non cambia l'array originale:

```
int[] numeri = {1, 2, 3};

for (int numero : numeri) {
    numero = numero * 2;
}

for (int numero : numeri) {
    System.out.println(numero);
}
```

Output:

```
1
2
3
```

numero è una copia temporanea del valore.

Se vuoi modificare l'array, usa l'indice:

```
for (int i = 0; i < numeri.length; i++) {  
    numeri[i] = numeri[i] * 2;  
}
```

Regola pratica

Usa for-each quando devi leggere tutti gli elementi.

Usa `for` con indice quando devi conoscere o modificare la posizione.

Ciclo for

Come ripetere istruzioni quando sai quante volte eseguirle.

Ripetere un'azione

Un ciclo serve a ripetere lo stesso blocco di codice.

Il ciclo `for` è utile quando sai quante volte vuoi ripetere.

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```


Le tre parti del for

Il ciclo ha questa forma:

```
for (inizializzazione; condizione; aggiornamento) {  
    // codice da ripetere  
}
```

Nell'esempio:

```
for (int i = 1; i <= 5; i++) {
```

succedono tre cose:

- `int i = 1` crea il contatore
- `i <= 5` dice fin quando continuare
- `i++` aumenta `i` di 1 dopo ogni giro

Contare da zero

In programmazione si conta spesso da zero, soprattutto con array e liste.

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Giro " + i);  
}
```

Output:

```
Giro 0  
Giro 1  
Giro 2
```

La condizione `i < 3` produce tre giri: 0, 1, 2.

Usare il for per sommare

```
int totale = 0;

for (int i = 1; i <= 5; i++) {
    totale = totale + i;
}

System.out.println(totale);
```

Output:

15

Il programma somma:

1 + 2 + 3 + 4 + 5

Attraversare un array

Un uso molto comune del `for` è leggere gli elementi di un array.

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(nomi[i]);
}
```

Output:

```
Luca  
Sara  
Mina
```

`nomi.length` contiene il numero di elementi dell'array.

Gli indici partono da `0`, quindi l'ultimo indice è `length - 1`.

Errore comune: andare oltre l'ultimo elemento

Questo codice è sbagliato:

```
String[] nomi = {"Luca", "Sara", "Mina"};  
  
for (int i = 0; i <= nomi.length; i++) {  
    System.out.println(nomi[i]);  
}
```

La condizione `i <= nomi.length` permette a `i` di arrivare a `3`, ma gli indici validi sono `0`, `1`, `2`.

La forma corretta è:

```
for (int i = 0; i < nomi.length; i++) {  
    System.out.println(nomi[i]);  
}
```

Ciclo all'indietro

Puoi anche contare al contrario:

```
for (int i = 5; i >= 1; i--) {  
    System.out.println(i);  
}
```

Output:

```
5
4
3
2
1
```

Quando usare for

Usa `for` quando:

- conosci il numero di ripetizioni
- hai un contatore
- vuoi attraversare un array con gli indici

Se invece vuoi ripetere finché una condizione resta vera, spesso è più naturale usare `while` .

Ciclo while

Come ripetere istruzioni finché una condizione resta vera.

Ripetere finché una condizione è vera

Il ciclo `while` ripete un blocco finché una condizione resta vera.

```
int numero = 1;

while (numero <= 3) {
    System.out.println(numero);
    numero++;
}
```

Output:

```
1  
2  
3
```

Java controlla la condizione prima di ogni giro.

Le parti di un while

Un ciclo `while` ha questa forma:

```
while (condizione) {  
    // codice da ripetere  
}
```

Nel nostro esempio:

```
while (numero <= 3)
```

il ciclo continua finché `numero` è minore o uguale a 3.

Dentro il ciclo aggiorniamo la variabile:

```
numero++;
```

Senza questo aggiornamento, `numero` resterebbe sempre uguale.

Il rischio del ciclo infinito

Questo codice non finisce mai:

```
int numero = 1;

while (numero <= 3) {
    System.out.println(numero);
}
```

`numero` resta sempre `1` , quindi la condizione e sempre vera.

Un ciclo che non finisce si chiama **ciclo infinito**.

Attenzione: quando scrivi un `while` , chiediti sempre: "dentro il ciclo, cosa cambia la condizione?"

Usare while con input utente

`while` e molto utile quando non sai prima quante volte l'utente fara qualcosa.

```
public class Password {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String password = "";

        while (!password.equals("java")) {
            System.out.print("Password: ");
            password = scanner.nextLine();
        }

        System.out.println("Accesso consentito.");
        scanner.close();
    }
}
```

Il ciclo continua finche la password non e `"java"` .

`!password.equals("java")` significa: "la password non e uguale a java".

Contare tentativi

Possiamo aggiungere un contatore:

```
public class Tentativi {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String password = "";
        int tentativi = 0;

        while (!password.equals("java")) {
            System.out.print("Password: ");
            password = scanner.nextLine();
            tentativi++;
        }

        System.out.println("Tentativi usati: " + tentativi);
        scanner.close();
    }
}
```

Ogni giro aumenta `tentativi` di 1.

while o for?

Usa `for` quando sai già quante volte ripetere:

```
for (int i = 0; i < 5; i++) {
    ...
}
```

Usa `while` quando vuoi ripetere finché succede qualcosa:

```
while (!password.equals("java")) {
    ...
}
```

Un ciclo che potrebbe non partire

Poiche `while` controlla la condizione all'inizio, il blocco potrebbe non essere eseguito nemmeno una volta.

```
int numero = 10;

while (numero < 5) {
    System.out.println(numero);
}
```

Qui non viene stampato nulla, perche la condizione e falsa gia all'inizio.

Metodi

Come dividere un programma Java in blocchi riutilizzabili.

Cosa e un metodo

Un metodo e un blocco di codice con un nome.

Lo scrivi una volta e puoi richiamarlo quando serve.

Pensa a un metodo come a una piccola ricetta: invece di riscrivere ogni passaggio, dai un nome all'azione.

Un primo metodo

```
public class Saluti {  
    public static void saluta() {  
        System.out.println("Ciao!");  
    }  
  
    public static void main(String[] args) {  
        saluta();  
        saluta();  
    }  
}
```

Output:

```
Ciao!  
Ciao!
```

Il metodo `saluta` viene definito una volta e chiamato due volte.

Definire e chiamare

Questa parte definisce il metodo:

```
public static void saluta() {  
    System.out.println("Ciao!");  
}
```

Questa parte lo chiama:

```
saluta();
```

Definire un metodo non lo esegue automaticamente. Viene eseguito quando lo chiami.

Perche usare metodi

I metodi aiutano a:

- evitare ripetizioni
- dare un nome a un'azione
- dividere un programma in pezzi piu piccoli
- rendere il codice piu facile da leggere

Senza metodo:

```
System.out.println("-----");
System.out.println("Prodotti");
System.out.println("-----");
System.out.println("Totale");
System.out.println("-----");
```

Con metodo:

```
public static void separatore() {
    System.out.println("-----");
}
```

Poi:

```
separatore();
System.out.println("Prodotti");
separatore();
System.out.println("Totale");
separatore();
```

Metodi statici nei primi programmi

Nei primi esempi useremo spesso:

```
public static void nomeMetodo()
```

`static` permette di chiamare il metodo direttamente da `main` , senza creare oggetti.

Piu avanti, nella programmazione a oggetti, vedremo metodi legati agli oggetti.

Nomi chiari

Il nome del metodo dovrebbe dire cosa fa.

```
public static void stampaTitolo() {  
    System.out.println("Lista della spesa");  
}
```

`stampaTitolo` e piu chiaro di `faiCosa` .

In Java i metodi usano di solito il camelCase:

```
calcolaTotale()  
mostraMessaggio()  
leggiPrezzo()
```

Esempio completo

```
public class Scontrino {  
    public static void stampaSeparatore() {  
        System.out.println("-----");  
    }  
  
    public static void main(String[] args) {  
        stampaSeparatore();  
        System.out.println("Pane");  
        System.out.println("Latte");  
        stampaSeparatore();  
        System.out.println("Totale: 3.20 euro");  
        stampaSeparatore();  
    }  
}
```

Un metodo non deve fare tutto. Di solito è meglio che abbia un compito preciso e piccolo.

Overloading

Come definire più metodi con lo stesso nome ma parametri diversi.

Lo stesso nome, parametri diversi

In Java puoi avere più metodi con lo stesso nome, se hanno parametri diversi.

Questa tecnica si chiama **overloading**.

```
public static void saluta() {  
    System.out.println("Ciao!");  
}  
  
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

Java sceglie quale metodo chiamare guardando gli argomenti.

Un primo esempio completo

```
public class Saluti {  
    public static void saluta() {  
        System.out.println("Ciao!");  
    }  
  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome + "!");  
    }  
  
    public static void main(String[] args) {  
        saluta();  
        saluta("Luca");  
    }  
}
```

Output:

```
Ciao!  
Ciao, Luca!
```

La prima chiamata non passa argomenti. La seconda passa una `String`.

La firma del metodo

La **firma** di un metodo è composta da:

- nome del metodo
- tipi dei parametri
- ordine dei parametri

Questi metodi hanno firme diverse:

```
public static void stampa(String testo) {  
    System.out.println(testo);  
}  
  
public static void stampa(int numero) {  
    System.out.println(numero);  
}
```

Uno riceve una `String`, l'altro un `int`.

Il valore di ritorno non basta

Non puoi distinguere due metodi solo dal tipo di ritorno.

Questo codice è sbagliato:

```
public static int calcola() {  
    return 1;  
}  
  
public static double calcola() {  
    return 1.0;  
}
```

Per Java le chiamate sarebbero ambigue:

```
calcola();
```

Quale dei due dovrebbe scegliere?

Parametri in ordine diverso

L'ordine conta.

```

public static void mostra(String nome, int eta) {
    System.out.println(nome + ", " + eta);
}

public static void mostra(int eta, String nome) {
    System.out.println(nome + ", " + eta);
}

```

Questi due metodi sono diversi per Java.

Pero non sempre e una buona idea: chi legge potrebbe confondersi. Usa l'overloading solo quando rende il codice piu chiaro.

Esempio: calcolare il totale

```

public class Totali {
    public static double totale(double prezzo, int quantita) {
        return prezzo * quantita;
    }

    public static double totale(double prezzo, int quantita, double
spedizione) {
        return prezzo * quantita + spedizione;
    }

    public static void main(String[] args) {
        System.out.println(totale(2.50, 4));
        System.out.println(totale(2.50, 4, 3.90));
    }
}

```

Output:

```

10.0
13.9

```

Il nome `totale` resta lo stesso perche l'idea e la stessa. Cambiano solo i dati disponibili.

Quando usarlo

L'overloading è utile quando:

- i metodi fanno la stessa azione generale
- cambiano solo i parametri disponibili
- il nome comune aiuta a leggere meglio

Evitalo se i metodi fanno cose davvero diverse. In quel caso scegli nomi diversi.

Parametri e valori di ritorno

Come passare dati a un metodo e ricevere un risultato.

Rendere un metodo più flessibile

Un metodo senza parametri fa sempre la stessa cosa.

```
public static void saluta() {  
    System.out.println("Ciao, Luca!");  
}
```

Se vogliamo salutare nomi diversi, usiamo un **parametro**.

Parametri

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

`String nome` è un parametro: una variabile che il metodo riceve quando viene chiamato.


```
saluta("Luca");  
saluta("Sara");
```

Output:

```
Ciao, Luca!  
Ciao, Sara!
```

I valori passati al metodo si chiamano **argomenti**.

Piu parametri

Un metodo puo ricevere piu parametri.

```
public static void mostraProdotto(String nome, double prezzo) {  
    System.out.println(nome + ": " + prezzo + " euro");  
}
```

Chiamata:

```
mostraProdotto("Pane", 1.50);  
mostraProdotto("Latte", 1.20);
```

Output:

```
Pane: 1.5 euro  
Latte: 1.2 euro
```

L'ordine degli argomenti deve corrispondere all'ordine dei parametri.

void

Finora abbiamo usato `void` .

```
public static void saluta(String nome)
```

`void` significa: questo metodo non restituisce un valore.

Il metodo fa qualcosa, per esempio stampa un messaggio, ma non consegna un risultato a chi lo chiama.

return

Se un metodo deve produrre un risultato, devi indicare il tipo del valore restituito.

```
public static double calcolaTotale(double prezzo, int quantita) {  
    return prezzo * quantita;  
}
```

Qui il metodo restituisce un `double` .

Puoi usare il risultato così:

```
double totale = calcolaTotale(2.50, 4);  
System.out.println(totale);
```

Output:

```
10.0
```

Il tipo di ritorno deve corrispondere

Se dichiari che un metodo restituisce `int` , deve restituire un `int` .

```
public static int doppio(int numero) {  
    return numero * 2;  
}
```

Questo invece è sbagliato:

```
public static int saluto() {  
    return "Ciao"; // errore  
}
```

Il metodo promette un `int`, ma prova a restituire una `String`.

Esempio completo

```
public class Calcoli {  
    public static double calcolaTotale(double prezzo, int quantita) {  
        return prezzo * quantita;  
    }  
  
    public static void mostraTotale(String prodotto, double totale) {  
        System.out.println(prodotto + ": " + totale + " euro");  
    }  
  
    public static void main(String[] args) {  
        double totalePane = calcolaTotale(1.50, 2);  
        double totaleLatte = calcolaTotale(1.20, 3);  
  
        mostraTotale("Pane", totalePane);  
        mostraTotale("Latte", totaleLatte);  
    }  
}
```

Output:

```
Pane: 3.0 euro
Latte: 3.5999999999999996 euro
```

Il secondo numero mostra una piccola imprecisione tipica dei numeri decimali nei computer. Per ora ci basta sapere che può succedere.

Regola pratica

Usa parametri quando il metodo ha bisogno di dati dall'esterno.

Usa `return` quando il metodo deve calcolare o preparare un valore da usare dopo.

Scope

Dove esistono le variabili e perché alcune non sono visibili ovunque.

Cosa significa scope

Lo **scope** è la zona del programma in cui una variabile esiste ed è visibile.

Una variabile non vive ovunque. Di solito vive solo dentro il blocco in cui è stata creata.

Variabili locali

Una variabile creata dentro un metodo si chiama variabile locale.

```
public static void saluta() {
    String nome = "Luca";
    System.out.println(nome);
}
```

`nome` esiste solo dentro `saluta`.

Questo codice è sbagliato:

```
public static void saluta() {  
    String nome = "Luca";  
}  
  
public static void main(String[] args) {  
    System.out.println(nome); // errore  
}
```

`main` non vede la variabile `nome` .

Parametri

Anche i parametri esistono solo dentro il metodo.

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome);  
}
```

`nome` è visibile dentro `saluta` , ma non fuori.

Se vuoi usare un valore in un metodo, passalo come parametro.

Blocchi con parentesi graffe

Anche un blocco `if` o `for` crea una zona.

```
if (true) {  
    int numero = 10;  
    System.out.println(numero);  
}  
  
// System.out.println(numero); // errore
```

`numero` nasce dentro il blocco `if` e sparisce alla fine del blocco.

Variabili create nel for

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
  
// System.out.println(i); // errore
```

La variabile `i` esiste solo dentro il ciclo.

Questo è utile: evita che contatori temporanei restino in giro nel resto del programma.

Due variabili con lo stesso nome

Non puoi dichiarare due variabili con lo stesso nome nello stesso blocco.

```
int eta = 20;  
int eta = 21; // errore
```

Puoi invece modificare la variabile:

```
int eta = 20;  
eta = 21;
```

Campi di classe

Più avanti useremo variabili dichiarate dentro una classe, ma fuori dai metodi. Si chiamano **campi**.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Questi campi appartengono agli oggetti della classe `Persona` .

Per ora ti basta distinguere:

- variabili locali: dentro un metodo o blocco
- parametri: valori ricevuti da un metodo
- campi: dati di un oggetto

Esempio corretto con parametri

Se un metodo deve usare un valore, passalo:

```
public class EsempioScope {  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome);  
    }  
  
    public static void main(String[] args) {  
        String nomeUtente = "Sara";  
        saluta(nomeUtente);  
    }  
}
```

Output:

```
Ciao, Sara
```

`nomeUtente` esiste in `main` . Il suo valore viene passato a `saluta` , dove arriva nel parametro `nome` .

Regola pratica

Quando Java dice che una variabile "cannot be resolved", spesso significa che stai provando a usarla fuori dal suo scope.

Controlla dove è stata dichiarata e dentro quali parentesi graffe si trova.

File

Come leggere e scrivere file di testo in modo semplice.

Perche usare i file

Finora i dati vivevano solo mentre il programma era aperto. Quando il programma finiva, le variabili sparivano.

Un file permette di salvare informazioni su disco: una lista, un messaggio, un risultato, una configurazione.

In questa pagina usiamo file di testo semplici.

Scrivere un file

Per scrivere testo in un file, puoi usare `FileWriter` .

```
public class ScriviFile {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("messaggio.txt");
            writer.write("Ciao dal file!");
            writer.close();

            System.out.println("File scritto.");
        } catch (IOException e) {
            System.out.println("Errore durante la scrittura.");
        }
    }
}
```

Dopo l'esecuzione, nella stessa cartella del programma troverai `messaggio.txt` .

Perche c'e try catch

Scrivere su un file puo fallire: la cartella potrebbe non esistere, il file potrebbe essere bloccato, oppure potresti non avere i permessi.

Java ti chiede di gestire questa possibilita.

Per ora leggi `try catch` così:

- prova a fare questa operazione
- se qualcosa va storto, esegui il blocco `catch`

Vedremo le eccezioni con più calma nella sezione sugli errori.

Leggere un file

Per leggere un file riga per riga, puoi usare `Scanner`.

Prima crea un file `messaggio.txt` con questo contenuto:

```
Ciao dal file!
```

Poi scrivi:

```
public class LeggiFile {
    public static void main(String[] args) {
        try {
            File file = new File("messaggio.txt");
            Scanner reader = new Scanner(file);

            while (reader.hasNextLine()) {
                String riga = reader.nextLine();
                System.out.println(riga);
            }

            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File non trovato.");
        }
    }
}
```

Output:

```
Ciao dal file!
```

Percorsi relativi

Nel codice abbiamo scritto:

```
new File("messaggio.txt")
```

Questo è un **percorso relativo**. Java cerca il file nella cartella da cui esegui il programma.

Se il programma dice "File non trovato", controlla:

- il nome del file
- l'estensione `.txt`
- la cartella in cui stai eseguendo `java`

Aggiungere testo a un file

Di solito `FileWriter` sovrascrive il file. Se vuoi aggiungere testo alla fine, usa il secondo parametro `true` :

```
FileWriter writer = new FileWriter("diario.txt", true);  
writer.write("Nuova riga\n");  
writer.close();
```

`\n` indica un a capo.

Regola pratica

Quando lavori con i file, ricordati tre cose:

1. le operazioni possono fallire, quindi servono `try catch`
2. le risorse aperte vanno chiuse
3. il percorso del file dipende dalla cartella in cui esegui il programma

I file sono il primo passo per far comunicare Java con dati esterni al codice.

Input da tastiera

Come leggere dati inseriti dall'utente con Scanner.

Leggere quello che scrive l'utente

L'input è ciò che entra nel programma. Nei primi esempi leggeremo dati dalla tastiera.

Per farlo useremo `Scanner`, una classe della libreria standard di Java.

Prima di usarla, devi importarla:

Un primo esempio

```
public class Saluto {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Come ti chiami? ");  
        String nome = scanner.nextLine();  
  
        System.out.println("Ciao, " + nome + "!");  
    }  
}
```

Esempio di esecuzione:

```
Come ti chiami? Luca  
Ciao, Luca!
```

`System.in` rappresenta l'input che arriva dal terminale.

`nextLine()` legge una riga intera di testo.

Leggere un numero intero

Per leggere un `int`, usa `nextInt()` :

```
public class Eta {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Quanti anni hai? ");  
        int eta = scanner.nextInt();  
  
        System.out.println("Tra un anno avrai " + (eta + 1) + " anni.");  
    }  
}
```

Esempio:

```
Quanti anni hai? 20  
Tra un anno avrai 21 anni.
```

Leggere un numero decimale

Per un numero con decimali, usa `nextDouble()` :

```

public class Prezzo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Prezzo: ");
        double prezzo = scanner.nextDouble();

        System.out.println("Doppio prezzo: " + (prezzo * 2));
    }
}

```

Nota: in molti ambienti Java si aspetta il punto per i decimali, per esempio `12.50` , non `12,50` .

Il piccolo problema dopo nextInt

Un errore comune nasce quando usi `nextInt()` e subito dopo `nextLine()` .

```

public class Profilo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Eta: ");
        int eta = scanner.nextInt();

        System.out.print("Nome: ");
        String nome = scanner.nextLine();

        System.out.println(nome + ", " + eta);
    }
}

```

Il programma potrebbe saltare la richiesta del nome. Succede perché `nextInt()` legge il numero, ma lascia nel buffer il carattere di a capo.

Una soluzione semplice è consumare quella riga prima di leggere il nome:

```
System.out.print("Eta: ");  
int eta = scanner.nextInt();  
scanner.nextLine(); // consuma l'a capo rimasto  
  
System.out.print("Nome: ");  
String nome = scanner.nextLine();
```

Chiudere lo Scanner

Negli esempi piccoli spesso vedrai:

```
scanner.close();
```

Chiudere lo scanner libera la risorsa usata per leggere.

In programmi brevi da terminale puoi metterlo alla fine:

```
Scanner scanner = new Scanner(System.in);  
  
// letture...  
  
scanner.close();
```

Esempio completo

```
public class Ordine {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Prodotto: ");  
        String prodotto = scanner.nextLine();  
  
        System.out.print("Quantita: ");  
        int quantita = scanner.nextInt();  
  
        System.out.print("Prezzo: ");  
        double prezzo = scanner.nextDouble();  
  
        double totale = quantita * prezzo;  
        System.out.println("Totale per " + prodotto + ": " + totale + "  
euro");  
  
        scanner.close();  
    }  
}
```

L'input rende il programma meno fisso: invece di usare sempre gli stessi valori, puoi farli scegliere all'utente.

Output

Come stampare testo e valori con System.out.

Mostrare informazioni nel terminale

L'output è ciò che il programma mostra all'esterno. Nei primi esercizi useremo il terminale.

In Java si stampa con `System.out`.

```
System.out.println("Ciao!");
```

Output:

```
Ciao!
```

println

`println` stampa un valore e poi va a capo.

```
public class Output {  
    public static void main(String[] args) {  
        System.out.println("Prima riga");  
        System.out.println("Seconda riga");  
    }  
}
```

Output:

```
Prima riga  
Seconda riga
```

Ogni chiamata a `println` produce una riga nuova.

print

`print` stampa senza andare a capo.


```
System.out.print("Ciao ");  
System.out.print("Luca");  
System.out.print("!");
```

Output:

```
Ciao Luca!
```

La differenza è piccola ma importante:

- `println` stampa e va a capo
- `print` stampa e resta sulla stessa riga

Stampare variabili

Puoi stampare il valore di una variabile:

```
String nome = "Sara";  
int eta = 21;  
  
System.out.println(nome);  
System.out.println(eta);
```

Output:

```
Sara  
21
```

Concatenare testo e valori

Per costruire un messaggio, usa `+`.

```
String nome = "Sara";  
int eta = 21;  
  
System.out.println("Nome: " + nome);  
System.out.println("Eta: " + eta);
```

Output:

```
Nome: Sara  
Eta: 21
```

Quando uno dei pezzi è una stringa, Java trasforma anche gli altri valori in testo.

Attenzione ai calcoli dentro le stringhe

Guarda questo esempio:

```
System.out.println("Totale: " + 10 + 5);
```

Output:

```
Totale: 105
```

Java parte da sinistra. Appena trova una stringa, unisce tutto come testo.

Se vuoi fare prima il calcolo, usa le parentesi:

```
System.out.println("Totale: " + (10 + 5));
```

Output:

Totale: 15

Formattare valori semplici

Per stampare un numero decimale con due cifre, puoi usare `printf`.

```
double prezzo = 12.5;
System.out.printf("Prezzo: %.2f euro%n", prezzo);
```

Output:

Prezzo: 12.50 euro

Qui `%.2f` significa: "stampa un numero decimale con due cifre dopo il punto".

`%n` manda a capo.

Esempio completo

```
public class Scontrino {
    public static void main(String[] args) {
        String prodotto = "Quaderno";
        int quantita = 3;
        double prezzo = 2.50;
        double totale = quantita * prezzo;

        System.out.println("Prodotto: " + prodotto);
        System.out.println("Quantita: " + quantita);
        System.out.printf("Totale: %.2f euro%n", totale);
    }
}
```

Output:

```
Prodotto: Quaderno  
Quantita: 3  
Totale: 7.50 euro
```

L'output e il modo piu semplice per controllare cosa sta facendo un programma.

Classi e oggetti

Come rappresentare entita del programma con classi, campi e oggetti.

L'idea della programmazione a oggetti

Java e un linguaggio molto legato alla programmazione a oggetti.

Un **oggetto** rappresenta qualcosa con dati e comportamenti: una persona, un prodotto, un conto, un libro.

Una **classe** e il modello da cui crei gli oggetti.

Pensala cosi:

- la classe e il modulo vuoto
- l'oggetto e un modulo compilato con dati reali

Creare una classe

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Questa classe descrive una persona con due campi:

- `nome`
- `eta`

I campi sono variabili che appartengono a un oggetto.

Creare un oggetto con new

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona();  
  
        persona.nome = "Luca";  
        persona.eta = 20;  
  
        System.out.println(persona.nome);  
        System.out.println(persona.eta);  
    }  
}
```

Output:

```
Luca  
20
```

`new Persona()` crea un nuovo oggetto partendo dalla classe `Persona` .

Classe e oggetto non sono la stessa cosa

La classe è il progetto.

L'oggetto è una cosa concreta creata da quel progetto.

```
Persona persona1 = new Persona();  
Persona persona2 = new Persona();  
  
persona1.nome = "Luca";  
persona2.nome = "Sara";
```

`persona1` e `persona2` sono due oggetti diversi. Hanno la stessa struttura, ma possono contenere dati diversi.

Aggiungere un metodo

Una classe può contenere anche metodi.

```
public class Persona {  
    String nome;  
    int eta;  
  
    void saluta() {  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

Ora puoi chiamare il metodo sull'oggetto:

```
Persona persona = new Persona();  
persona.nome = "Luca";  
  
persona.saluta();
```

Output:

```
Ciao, sono Luca
```

Il metodo usa il campo `nome` dell'oggetto su cui viene chiamato.

Un esempio con prodotti

```
public class Prodotto {  
    String nome;  
    double prezzo;  
  
    void mostra() {  
        System.out.println(nome + ": " + prezzo + " euro");  
    }  
}
```

Uso:

```
public class Negozio {  
    public static void main(String[] args) {  
        Prodotto prodotto = new Prodotto();  
        prodotto.nome = "Quaderno";  
        prodotto.prezzo = 2.50;  
  
        prodotto.mostra();  
    }  
}
```

Output:

```
Quaderno: 2.5 euro
```

Perche usare classi

Le classi aiutano a tenere insieme dati collegati.

Senza classe potresti avere variabili separate:

```
String nomeProdotto = "Quaderno";  
double prezzoProdotto = 2.50;
```

Con una classe, il concetto diventa più chiaro:

```
Prodotto prodotto = new Prodotto();
```

Man mano che i programmi crescono, questa organizzazione diventa fondamentale.

Costruttori

Come inizializzare un oggetto quando viene creato.

Il problema da risolvere

Nella pagina precedente creavamo un oggetto e poi riempivamo i campi:

```
Persona persona = new Persona();  
persona.nome = "Luca";  
persona.eta = 20;
```

Funziona, ma è facile dimenticare un campo.

Un **costruttore** serve a inizializzare l'oggetto nel momento in cui viene creato.

Creare un costruttore

```
public class Persona {  
    String nome;  
    int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
}
```

Il costruttore ha lo stesso nome della classe.

Non ha tipo di ritorno: non scrivi ne `void` ne `int` ne altro.

Usare il costruttore

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona("Luca", 20);  
  
        System.out.println(persona.nome);  
        System.out.println(persona.eta);  
    }  
}
```

Output:

```
Luca  
20
```

I valori `"Luca"` e `20` vengono passati al costruttore.

A cosa serve this

Nel costruttore abbiamo scritto:

```
this.nome = nome;
```

Qui ci sono due `nome` :

- `this.nome` e il campo dell'oggetto
- `nome` e il parametro del costruttore

`this` significa "questo oggetto".

La riga si legge così: "metti nel campo `nome` di questo oggetto il valore ricevuto nel parametro `nome`".

Costruttore predefinito

Se non scrivi nessun costruttore, Java ne crea uno vuoto per te.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Puoi fare:

```
Persona persona = new Persona();
```

Ma appena scrivi un costruttore con parametri, Java non crea più automaticamente quello vuoto.

Piu costruttori

Puoi avere più costruttori con parametri diversi.

```

public class Persona {
    String nome;
    int eta;

    public Persona() {
        nome = "Sconosciuto";
        eta = 0;
    }

    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}

```

Uso:

```

Persona a = new Persona();
Persona b = new Persona("Sara", 25);

```

Questo è un caso di overloading applicato ai costruttori.

Esempio completo

```

public class Prodotto {
    String nome;
    double prezzo;

    public Prodotto(String nome, double prezzo) {
        this.nome = nome;
        this.prezzo = prezzo;
    }

    void mostra() {
        System.out.println(nome + ": " + prezzo + " euro");
    }
}

```

```
public class Negozio {  
    public static void main(String[] args) {  
        Prodotto quaderno = new Prodotto("Quaderno", 2.50);  
        Prodotto penna = new Prodotto("Penna", 1.20);  
  
        quaderno.mostra();  
        penna.mostra();  
    }  
}
```

I costruttori rendono più difficile creare oggetti incompleti.

Incapsulamento

Come proteggere i dati di un oggetto con private, getter e setter.

Perché non lasciare tutto pubblico

Finora abbiamo modificato i campi direttamente:

```
persona.eta = -5;
```

Java lo permette se il campo è accessibile, ma questo valore non ha senso.

L'**incapsulamento** serve a proteggere i dati di un oggetto e controllare come vengono letti o modificati.

private

Con `private`, un campo può essere usato solo dentro la classe.

```
public class Persona {  
    private String nome;  
    private int eta;  
}
```

Da fuori non puoi più fare:

```
persona.eta = 20; // errore se eta e private
```

Serve un modo controllato per leggere e modificare il valore.

Getter

Un getter restituisce il valore di un campo.

```
public String getNome() {  
    return nome;  
}
```

Esempio:

```
public class Persona {  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
}
```

`getNome` permette di leggere `nome` senza rendere il campo pubblico.

Setter

Un setter modifica il valore di un campo.

```
public void setName(String nome) {  
    this.nome = nome;  
}
```

Il vantaggio è che puoi controllare il valore prima di salvarlo.

```
public void setEta(int eta) {  
    if (eta >= 0) {  
        this.eta = eta;  
    }  
}
```

Se qualcuno prova a impostare `-5`, il valore non viene accettato.

Classe completa

```
public class Persona {  
    private String nome;  
    private int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        setEta(eta);  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public int getEta() {  
        return eta;  
    }  
  
    public void setEta(int eta) {  
        if (eta >= 0) {  
            this.eta = eta;  
        }  
    }  
}
```

Uso:

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona("Luca", 20);  
  
        System.out.println(persona.getNome());  
        System.out.println(persona.getEta());  
  
        persona.setEta(21);  
        System.out.println(persona.getEta());  
    }  
}
```

Output:

```
Luca
20
21
```

public e private

`public` significa accessibile dall'esterno.

`private` significa accessibile solo dentro la classe.

Una regola molto comune in Java e:

- campi `private`
- metodi pubblici solo quando servono

Perche e utile

L'incapsulamento aiuta a:

- evitare valori non validi
- cambiare l'interno della classe senza rompere tutto il programma
- rendere chiaro quali azioni sono permesse

All'inizio sembra piu lungo. Nei programmi reali rende il codice piu sicuro e ordinato.

Ereditarieta

Come creare classi specializzate partendo da classi piu generali.

Cosa significa ereditarieta

L'ereditarieta permette di creare una classe partendo da un'altra.

La classe piu generale contiene dati e metodi comuni. La classe piu specifica li riusa e aggiunge cio che le serve.

Esempio:

- `Persona` e generale

- `Studente` è una persona con una matricola

extends

In Java si usa `extends` .

```
public class Persona {  
    String nome;  
  
    void saluta() {  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

```
public class Studente extends Persona {  
    String matricola;  
}
```

`Studente` eredita il campo `nome` e il metodo `saluta` .

Usare la classe derivata

```
public class Esempio {  
    public static void main(String[] args) {  
        Studente studente = new Studente();  
  
        studente.nome = "Luca";  
        studente.matricola = "A123";  
  
        studente.saluta();  
        System.out.println(studente.matricola);  
    }  
}
```

Output:

Ciao, sono Luca
A123

Classe base e classe derivata

La classe da cui parti viene spesso chiamata:

- classe base
- superclasse
- classe padre

La classe che eredita viene chiamata:

- classe derivata
- sottoclasse
- classe figlia

Sono parole diverse per descrivere la stessa relazione.

Sovrascrivere un metodo

Una sottoclasse puo ridefinire un metodo ereditato.

```
public class Studente extends Persona {  
    String matricola;  
  
    @Override  
    void saluta() {  
        System.out.println("Ciao, sono lo studente " + nome);  
    }  
}
```

`@Override` dice a Java e a chi legge: "sto sovrascrivendo un metodo della classe base".

Costruttori e super

Se la classe base ha un costruttore con parametri, la sottoclasse deve chiamarlo con `super`.

```
public class Persona {
    private String nome;

    public Persona(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

```
public class Studente extends Persona {
    private String matricola;

    public Studente(String nome, String matricola) {
        super(nome);
        this.matricola = matricola;
    }
}
```

`super(nome)` chiama il costruttore di `Persona` .

Quando usarla con attenzione

L'ereditarietà è utile quando una classe è davvero un tipo più specifico di un'altra.

`Studente` è una `Persona` : ha senso.

Ma non usarla solo per "prendere codice". A volte è meglio mettere un oggetto dentro un altro invece di ereditarlo.

Regola pratica:

se puoi dire "X è un tipo di Y", l'ereditarietà potrebbe avere senso.

Se devi dire "X usa Y" o "X contiene Y", probabilmente serve un'altra struttura.

Interfacce

Come definire comportamenti comuni che classi diverse possono implementare.

Cosa è un'interfaccia

Un'interfaccia descrive un comportamento che una classe promette di avere.

Non dice necessariamente come farlo. Dice quali metodi devono esistere.

Pensala come un contratto:

"se una classe implementa questa interfaccia, allora deve offrire questi metodi".

Creare un'interfaccia

```
public interface Stampabile {  
    void stampa();  
}
```

Questa interfaccia richiede un metodo `stampa` .

Una classe che la implementa deve scrivere quel metodo.

implements

```
public class Documento implements Stampabile {  
    private String titolo;  
  
    public Documento(String titolo) {  
        this.titolo = titolo;  
    }  
  
    @Override  
    public void stampa() {  
        System.out.println("Documento: " + titolo);  
    }  
}
```

`implements Stampabile` significa: `Documento` rispetta il contratto `Stampabile`.

Classi diverse, stesso comportamento

```
public class Foto implements Stampabile {  
    private String nomeFile;  
  
    public Foto(String nomeFile) {  
        this.nomeFile = nomeFile;  
    }  
  
    @Override  
    public void stampa() {  
        System.out.println("Foto: " + nomeFile);  
    }  
}
```

`Documento` e `Foto` sono classi diverse, ma entrambe sanno stampare.

Usare il tipo interfaccia

Puoi usare l'interfaccia come tipo.

```
public class Esempio {
    public static void main(String[] args) {
        Stampabile a = new Documento("Contratto");
        Stampabile b = new Foto("vacanza.jpg");

        a.stampa();
        b.stampa();
    }
}
```

Output:

```
Documento: Contratto
Foto: vacanza.jpg
```

Il programma non ha bisogno di sapere tutti i dettagli interni. Gli basta sapere che entrambi sono `Stampabile` .

Perche sono utili

Le interfacce aiutano a scrivere codice flessibile.

Puoi dire: "mi serve qualcosa che sappia fare questa azione", senza fissarti su una classe specifica.

Esempio:

```
public static void stampaOggetto(Stampabile oggetto) {
    oggetto.stampa();
}
```

Questo metodo funziona con qualunque classe che implementa `Stampabile` .

Interfacce ed ereditarieta

Una classe Java puo estendere una sola classe:

```
public class Studente extends Persona {  
}
```

Ma puoi implementare più interfacce:

```
public class Documento implements Stampabile, Salvabile {  
}
```

Questo rende le interfacce molto utili per descrivere comportamenti comuni.

Regola pratica

Usa un'interfaccia quando classi diverse devono condividere lo stesso comportamento, anche se non appartengono alla stessa famiglia.

`Documento` e `Foto` non sono la stessa cosa. Però entrambe possono essere stampabili.

Buone pratiche

Regole semplici per scrivere codice Java più leggibile e mantenibile.

Scrivere per farsi capire

Il codice viene letto molte più volte di quante venga scritto.

Anche quando lavori da solo, il "lettore futuro" sarai tu fra qualche settimana.

Le buone pratiche servono a rendere il codice più chiaro, non a renderlo più elegante per forza.

Usa nomi chiari

Meglio:

```
double prezzoTotale = prezzo + spedizione;
```

che:

```
double x = a + b;
```

Nomi come `x`, `a`, `b` vanno bene solo in esempi molto piccoli o calcoli evidenti.

Per variabili e metodi usa il camelCase:

```
nomeUtente  
calcolaTotale()  
mostraMessaggio()
```

Per classi usa la maiuscola iniziale:

```
Persona  
Prodotto  
Calcolatrice
```

Metodi piccoli

Un metodo dovrebbe fare una cosa principale.

Se un metodo:

- legge input
- calcola
- stampa
- salva su file

tutto insieme, diventa difficile da capire e da testare.

Prova a dividerlo:


```
public static double calcolaTotale(double prezzo, int quantita) {  
    return prezzo * quantita;  
}  
  
public static void mostraTotale(double totale) {  
    System.out.println("Totale: " + totale);  
}
```

Evita duplicazioni

Se ripeti lo stesso blocco piu volte, valuta un metodo.

Prima:

```
System.out.println("-----");  
System.out.println("Prodotti");  
System.out.println("-----");  
System.out.println("Totale");  
System.out.println("-----");
```

Dopo:

```
public static void separatore() {  
    System.out.println("-----");  
}
```

La duplicazione non e solo una questione di righe. Se devi cambiare qualcosa, rischi di dimenticare una copia.

Formatta il codice

Usa indentazione coerente.

```
if (eta >= 18) {  
    System.out.println("Maggiorenne");  
} else {  
    System.out.println("Minorenne");  
}
```

Evita codice tutto attaccato:

```
if(eta>=18){System.out.println("Maggiorenne");}
```

Il computer lo capisce, ma le persone fanno piu fatica.

Non ottimizzare troppo presto

All'inizio scrivi codice chiaro.

Non cercare la versione piu corta o piu furba se rende il programma difficile da leggere.

Prima chiediti:

- funziona?
- e chiaro?
- e facile da modificare?

Solo dopo ha senso pensare alle prestazioni, se c'e un problema reale.

Commenti utili

Un commento deve spiegare cio che il codice non rende evidente.

Utile:

```
// Applichiamo lo sconto solo agli ordini sopra 50 euro  
if (totale > 50) {  
    totale = totale - 5;  
}
```

Poco utile:

```
// Aggiunge 5 a totale  
totale = totale + 5;
```

Regola pratica

Quando finisci un programma, rileggilo come se fosse di un'altra persona.

Se una riga ti costringe a fermarti troppo, forse puoi:

- rinominare una variabile
- estrarre un metodo
- aggiungere una nota breve
- dividere un'espressione in due passaggi

Il codice buono non è quello che sembra difficile. È quello che puoi capire, correggere e far crescere.

Maven e Gradle

A cosa servono gli strumenti di build nei progetti Java.

Perché servono strumenti di build

Nei primi esercizi compili con:

```
javac NomeFile.java
```

Funziona bene per programmi piccoli.

Nei progetti reali ci sono molte classi, librerie esterne, test e file di configurazione. Gestire tutto a mano diventa scomodo.

Qui entrano in gioco Maven e Gradle.

Cosa fanno Maven e Gradle

Uno strumento di build puo:

- compilare il progetto
- scaricare librerie esterne
- eseguire test
- creare un file pronto da distribuire
- mantenere una struttura ordinata

Le librerie esterne si chiamano spesso **dipendenze**: pezzi di codice scritti da altri che il tuo progetto usa.

Maven

Maven usa un file chiamato `pom.xml` .

Una struttura tipica e:

```
progetto/  
  pom.xml  
  src/  
    main/  
      java/  
    test/  
      java/
```

Il codice principale va in:

```
src/main/java
```

I test vanno in:

```
src/test/java
```

Comandi comuni:

```
mvn compile
mvn test
mvn package
```

Gradle

Gradle usa spesso un file `build.gradle` oppure `build.gradle.kts`.

Anche Gradle usa una struttura simile:

```
progetto/
  build.gradle
  src/
    main/
      java/
    test/
      java/
```

Comandi comuni:

```
gradle build
gradle test
```

In molti progetti troverai uno script incluso:

```
./gradlew build
```

Su Windows:

```
gradlew.bat build
```

Questo permette di usare Gradle senza installarlo manualmente.

Dipendenze

Se vuoi usare una libreria esterna, la dichiari nel file di build.

Non devi scaricare manualmente file `.jar` e copiarli in giro.

Lo strumento di build legge la configurazione, scarica cio che serve e lo collega al progetto.

Quando ti servono davvero

All'inizio puoi continuare con `javac` e `java`.

Maven o Gradle diventano utili quando:

- il progetto ha molte classi
- vuoi usare librerie esterne
- vuoi scrivere test automatici
- lavori con altre persone
- usi framework come Spring

Regola pratica

Per imparare la sintassi Java, parti semplice.

Quando il programma inizia ad avere piu cartelle, librerie e test, passa a Maven o Gradle.

Non sono "Java avanzato" nel senso del linguaggio, ma sono parte della vita quotidiana di molti progetti Java.

Package e import

Come organizzare classi in package e usare codice da altre librerie.

Perche organizzare il codice

Quando un progetto contiene poche classi, puoi tenerle nella stessa cartella.

Quando cresce, serve ordine. I **package** aiutano a raggruppare classi collegate.

Un package e come una cartella logica per il codice.

Dichiarare un package

In cima a un file Java puoi scrivere:

```
package negozio;
```

Poi definisci la classe:

```
package negozio;

public class Prodotto {
    private String nome;

    public Prodotto(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

Di solito la cartella rispecchia il package:

```
negozio/Prodotto.java
```

Usare import

Se una classe si trova in un package diverso, puoi importarla.

Questo import ti permette di scrivere:

```
ArrayList nomi = new ArrayList<>();
```

Senza import dovresti scrivere il nome completo:

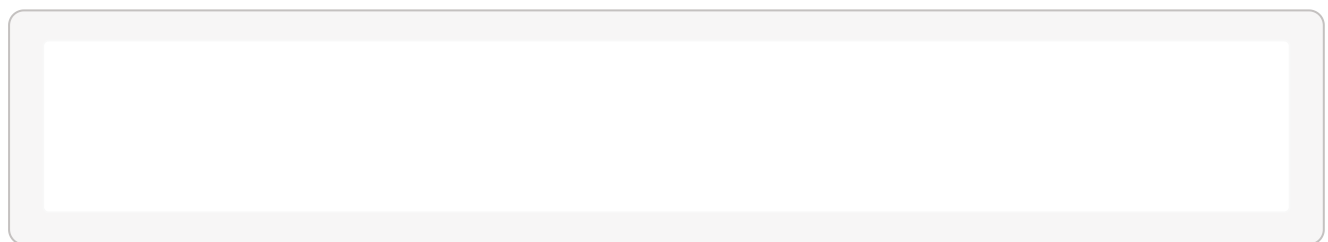
```
java.util.ArrayList nomi = new java.util.ArrayList<>();
```

Troppo lungo per l'uso quotidiano.

Import dalla libreria standard

Molte classi utili fanno parte della libreria standard Java.

Esempi:



`java.util` contiene molte strutture e strumenti generali.

`java.io` contiene classi per input e output, inclusi i file.

Importare una propria classe

Immagina questa struttura:

```
src/  
  negozio/  
    Prodotto.java  
  app/  
    Main.java
```

`Prodotto.java` :


```
package negozio;

public class Prodotto {
}
```

Main.java :

```
package app;

public class Main {
    public static void main(String[] args) {
        Prodotto prodotto = new Prodotto();
    }
}
```

Errori comuni

Il package dichiarato deve essere coerente con la cartella.

Se un file dice:

```
package negozio;
```

ma lo tieni in una cartella non coerente, strumenti e compilatore possono confondersi.

Un altro errore comune è dimenticare `public` su una classe che vuoi usare da un altro package.

```
public class Prodotto {
}
```

Regola pratica

All'inizio usa package semplici e nomi chiari.

Quando un progetto cresce, raggruppa per argomento:

```
app
model
service
repository
```

Non creare package solo per complicare. Devono aiutare a trovare il codice.

Test

Come verificare il comportamento del codice con test automatici semplici.

Perche scrivere test

Un test automatico controlla che una parte del programma funzioni come previsto.

Invece di provare tutto a mano ogni volta, scrivi codice che verifica altro codice.

Questo e utile quando:

- modifichi una funzione
- vuoi evitare regressioni
- lavori su programmi piu grandi

Un metodo da testare

Immagina questa classe:

```
public class Calcolatrice {
    public static int somma(int a, int b) {
        return a + b;
    }
}
```

Vogliamo controllare che `somma(2, 3)` restituisca `5`.

L'idea di test unitario

Un **test unitario** controlla una piccola unità di codice, spesso un metodo.

Con JUnit, una libreria molto usata per i test Java, potresti scrivere:

```
public class CalcolatriceTest {  
    @Test  
    void sommaDueNumeri() {  
        int risultato = Calcolatrice.somma(2, 3);  
        assertEquals(5, risultato);  
    }  
}
```

`assertEquals(5, risultato)` significa: mi aspetto che `risultato` sia uguale a `5`.

Test che passa e test che fallisce

Se il metodo è corretto:

```
return a + b;
```

il test passa.

Se per errore scrivi:

```
return a - b;
```

il test fallisce e ti segnala che il risultato ottenuto non corrisponde a quello atteso.

Cosa testare

Inizia da metodi semplici che restituiscono un valore.

Esempi buoni:

```
calcolaTotale(2.50, 4)
isMaggiorenni(20)
formattaNome("luca")
```

Sono facili da controllare perché hanno input chiari e output chiari.

Evitare test troppo grandi

Un test dovrebbe controllare una cosa principale.

Meglio tre test piccoli:

- somma numeri positivi
- somma con zero
- somma numeri negativi

che un solo test lungo che controlla tutto insieme.

Test e strumenti di build

JUnit di solito si usa dentro un progetto Maven o Gradle.

Con Maven, il comando tipico è:

```
mvn test
```

Con Gradle:

```
gradle test
```

Questi strumenti compilano il codice, eseguono i test e mostrano il risultato.

Regola pratica

All'inizio non devi testare ogni riga.

Prova a testare i metodi che:

- fanno calcoli
- controllano condizioni
- trasformano dati
- possono rompersi quando li modifichi

I test non sostituiscono il ragionamento, ma ti danno una rete di sicurezza.